A Big Data Framework for Price Tracking and Product Matching in Sri Lankan E-Commerce

P.K.K. Dias

July 20, 2026

Contents

1 Introduction **4**

1.1 Background . . . 4

1.2 Problem statement . . . 5

2 Literature Review **6**

2.1 Anatomy and structure of websites and web pages . . . 6

2.1.1 Structure of a website . . . 6

2.1.2 Anatomy of a web page . . . 6

2.1.3 HTML tags and the document object model (DOM) . . . 7

2.1.4 Information hierarchy in e-commerce websites . . . 8

2.1.5 The semantic web and structured data . . . 8

2.2 Crawling the Web . . . 9

2.2.1 Summary . . . 11

2.3 Boilerplate detection and removal . . . 11

2.3.1 Site-level methods . . . 12

2.3.2 Page-level methods . . . 13

2.3.3 Performance improvements . . . 14

2.3.4 Summary . . . 14

2.4 Web page classification . . . 14

2.4.1 Web page classification techniques . . . 15

2.4.2 Techniques without negative examples . . . 16

2.4.3 Effect of noise removal . . . 16

2.4.4 MarkupLM . . . 17

2.4.5 E-commerce web page classification . . . 17

2.4.6 Summary . . . 17

2.5 Web information extraction . . . 18

2.5.1 Embedded semantic metadata . . . 19

2.5.2 Vision-based approaches . . . 20

2.5.3 DOM tree node classification . . . 21

2.5.4 Markup and layout-based approaches . . . 22

2.5.5 Multimodal approaches . . . 23

2.5.6 LLM-based approaches . . . 24

2.5.7 Summary . . . 24

2.6 Product matching . . . 25

2.6.1 Blocking and filtering techniques . . . 25

2.6.2 Verification techniques . . . 26

2.7 Related work . . . 29

2.7.1 Consumer-facing public systems . . . 29

2.7.2 Self-hosted and open-source systems . . . 30
2.7.3 Enterprise competitive intelligence systems . . . 31
2.7.4 Summary of Related Systems . . . 32

3 Problem Specification and Requirements 33
3.1 Problem Specification . . . 33
3.2 Aims and objectives . . . 33
3.2.1 Aims . . . 33
3.2.2 Objectives . . . 34
3.3 Requirements . . . 34
3.3.1 Functional requirements . . . 34
3.3.2 Non-functional requirements . . . 35

4 Methodology and Software Tools 36
4.1 Methodology . . . 36
4.1.1 Approach . . . 36
4.1.2 Evaluation . . . 37
4.2 Software Tools . . . 37
4.2.1 Web crawling and data collection . . . 38
4.2.2 Boilerplate removal . . . 39
4.2.3 Proof-of-concept application . . . 39
4.2.4 Data storage and management . . . 39
4.2.5 Containerization and orchestration . . . 40
4.2.6 Model training and inference . . . 40
4.2.7 Programming languages . . . 41
4.2.8 Infrastructure as Code (IaC) . . . 41
4.2.9 Source code management . . . 41
4.2.10 Continuous Integration and Continuous Deployment (CI/CD) . . . 42
4.2.11 Other tools . . . 42

5 System design 43
5.1 Database design . . . 44
5.1.1 Database schema . . . 44
5.2 Distributed crawler . . . 45
5.2.1 Crawler queue . . . 46
5.2.2 Queue manager . . . 46
5.2.3 Crawler workers . . . 46
5.2.4 Crawler system architecture . . . 49
5.3 HTML minimizer . . . 49
5.3.1 Boilerplate removal library . . . 50
5.3.2 Anchor tree generation . . . 53
5.3.3 HTML minimizer workers . . . 53
5.3.4 HTML minimizer architecture . . . 54
5.4 Data annotation application . . . 55
5.5 Page classifier . . . 56
5.5.1 Training phase . . . 56
5.5.2 Model evaluation and selection . . . 57

5.5.3 Inference phase . . . 58
5.5.4 Page classifier architecture . . . 58
5.6 Information extraction . . . 59
5.6.1 Training phase . . . 60
5.6.2 Model evaluation and selection . . . 64
5.6.3 Inference phase . . . 65
5.6.4 Price history tracking . . . 66
5.6.5 Information extraction module architecture . . . 67
5.7 Product matching . . . 68
5.7.1 Comparison of different approaches . . . 68
5.7.2 Candidate generation (filtering) phase . . . 70
5.7.3 Candidate ranking (verification) phase . . . 71
5.8 Proof-of-concept (PoC) system . . . 74

Chapter 1

Introduction

1.1 Background

Globally, e-commerce (or online shopping) has been a significant driver of revenue for business as well as a vital utility for consumers to access goods and services that may have otherwise been difficult to access. Total e-commerce sales across the globe was projected to surpass 6.86 trillion USD by the end of 2025 [**?**]. While western markets such as North America and Europe have traditionally dominated this sector, the focus has been rapidly shifting towards Asia. South Asia in particular has been a region of strong growth with a Compound Annual Growth Rate (CAGR) exceeding 20% [**?**].

Sri Lanka is a notable player in this rapidly growing subspace, with its digital economy being valued at 1,342 billion LKR in 2025, contributing 4.5% to the national GDP [**?**]. This growth was accelerated during the COVID-19 pandemic, which forced a migration of both consumers and SMEs to online shopping. Many of these e-commerce platforms offer consumers access to products that are otherwise unavailable in their localities, while also expanding the reach for domestic businesses [**?**]. However, a barrier for growth, is the lack of consumer trust in online shopping which is currently plagued by a "fake discount epidemic", with anti-consumer practices such as false reference pricing, perpetual sales, and bait and switch techniques [**?**].

Data from neighbouring countries such as India have shown that over 50% of consumers who engage in online shopping have fallen victim to such scams [**?**]. This pattern seems to be mirrored in Sri Lanka as well, where profit maximization is often prioritized over consumer satisfaction. Moreover, the lack of a centralized data source to compare product features and prices also prevents finding the best deal, often a very labour intensive and time-consuming process for the average user. Modern consumers are also aware of "dynamic pricing", which is the use of algorithms to dynamically adjust prices of products based on internal factors such as customer profiling as well as external trends such as demand and availability. Online shoppers are driven to monitor product prices and wait for markdowns before making a purchase [**?**]. From a data perspective, Sri Lanka's e-commerce landscape is highly fragmented.

This project aims to bridge this information gap by performing a comprehensive analysis of Sri Lanka's local e-commerce ecosystem, centered around consumer goods (which is the largest single category in the nation, with over 76% of e-commerce users have purchased consumer goods online at least once [**?**])), By leveraging techniques of big data analytics, we will also aim to identify pricing correlations with external events, understand the prevalence of the "fake discount epidemic", and develop a generalized model which is capable of automated data extractions. Ultimately, this work will facilitate the creation of tools for price

tracking and cross-site product comparison, empowering consumers and providing the data-driven insights necessary to foster innovation and transparency in Sri Lanka's digital economy.

1.2 Problem statement

Currently, hunting for deals and comparing prices across multiple online stores can be a time-consuming and tedious task for consumers. With the increasing number of retailers and e-commerce marketplaces offering their products to be purchased online, consumers often find themselves overwhelmed with the sheer volume of options available to them. Moreover, it is difficult for consumers to identify whether sales and discounts offered on these platforms are genuine or if they are simply marketing tactics to attract customers, where the original prices are marked up to create the illusion of a discount. With the same product offered at different platforms, consumers are often left wondering which platform offers the best deal, and whether they are getting the best value for their money.

While LLMs and agentic systems have to a certain extent, have proven useful in solving these problems, they do have their limitations.

- **Limited selection**: When prompted, LLMs do not provide results from all available platforms, but rather to a selection of 3-5 platforms, which may not be representative of the entire market. This can lead to consumers missing out on better deals available on other platforms.
- **Inability to track historical prices**: While they are able to access real-time data, LLMs are unable to track historical prices of products, which can be useful in determining whether a sale or discount is genuine or not.
- **High cost and resource usage**: LLMs are resource-intensive and can be costly to run, especially when dealing with large volumes of data. While for small-scale product comparison, LLMs may be feasible, for larger-scale comparisons, they may not be practical.

Therefore, there is a need for a more efficient and scalable solution that can provide consumers with accurate and comprehensive information about products and their prices across multiple online platforms, while also being able to track historical prices and identify genuine sales and discounts.

Chapter 2

Literature Review

2.1 Anatomy and structure of websites and web pages

2.1.1 Structure of a website

The structure of a web page generally follows one of four structural models [**?**].

1. **Hierarchical (tree structure)**: This is the most common model, mirroring human cognitive processes and the way we organize information. It consists of a root node (the homepage) with branches leading to category pages, which further branch out to product pages and other content. This structure is intuitive for users and allows for easy navigation.
2. **Sequential (linear structure)**: In this model, pages are arranged in a linear sequence, often used for storytelling or step-by-step guides. Users navigate through the content in a predetermined order, which can be effective for certain types of content but may limit user freedom. This style is common in e-commerce checkout processes, where users are guided through a series of steps to complete a purchase.
3. **Matrix (grid structure)**: This model allows for multiple pathways between pages, with links connecting various nodes in a non-linear fashion. It is often used in content-rich websites, such as news sites or social media platforms, where users can navigate through different topics and categories without a strict hierarchy.
4. **Database (dynamic structure)**: In this model, content is generated dynamically from a database based on user queries or interactions. It is common in e-commerce sites, where product pages are generated based on user searches or filters, and in social media platforms, where user-generated content is displayed based on relevance or popularity.

2.1.2 Anatomy of a web page

A web page is collection of elements that are common components of the overall website (site-level components), and elements that are specific to the individual page (page-level components) [**?**].

Site-level components

Site-level components are elements that are typically present across multiple pages of a website, providing consistency and aiding in navigation. These include:

- **Header**: The header is located at the top of the page and often contains the website's logo, navigation menu, search bar, and sometimes contact information or social media links. It serves as a consistent element across the site, allowing users to easily navigate to different sections of the website.
- **Footer**: The footer is located at the bottom of the page and often contains additional navigation links, contact information, copyright notices, and sometimes social media links. Like the header, it provides consistency across the site and can help users find important information or navigate to other sections.

Page-level components

Page-level components are elements that are specific to the individual page and can vary widely depending on the type of page and its purpose. These include:

- **Main content area**: This is the central part of the page where the primary content is displayed. It can include text, images, videos, and other media, and is the main focus of the page.
- **Sidebar**: A sidebar is an additional column that can be located on either side of the main content area. It often contains supplementary information, such as related articles, advertisements, or additional navigation links.
- **Hero section**: A hero section is a prominent area at the top of the page, often featuring a large image or video, along with a headline and call-to-action button. It is designed to capture the user's attention and convey the main message or purpose of the page.

2.1.3 HTML tags and the document object model (DOM)

HTML (HyperText Markup Language) is the standard language used to create and structure web pages. It uses a system of tags to define the structure and content of a web page, allowing browsers to render the page correctly. Semantic HTML involves the use of HTML tags such as `<header>`, `<nav>`, `<main>`, `<article>`, and `<footer>` to provide meaning to the content and structure of the page, which can improve accessibility and SEO (Search Engine Optimization).

The Document Object Model (DOM) is a programming interface for HTML and XML documents. It represents the structure of a web page as a tree of nodes, where each node corresponds to an element in the HTML document. The DOM allows developers to manipulate the content and structure of a web page dynamically using JavaScript, enabling interactive features and dynamic content updates without requiring a page reload [**?**].

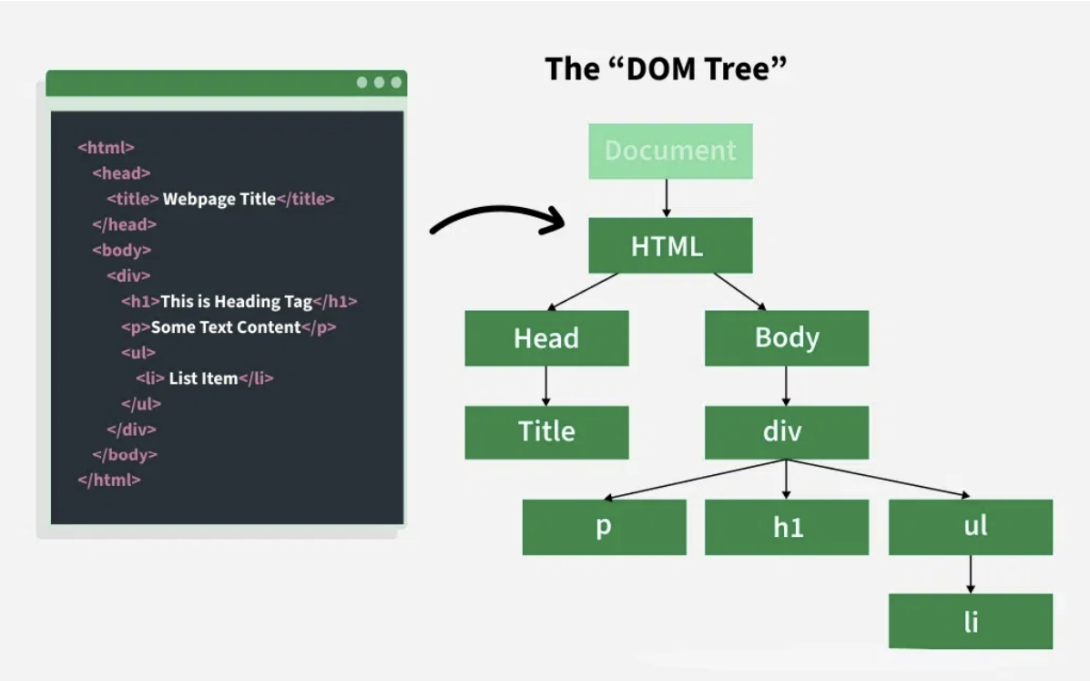

Figure 2.1: HTML and the corresponding DOM tree [**?**]

2.1.4 Information hierarchy in e-commerce websites

E-commerce sites are usually structured around a conversion funnel designed to move visitors from awareness to purchase [**?**].

- **Homepage**: The homepage serves as the entry point to the website and provides an overview of the site's offerings, often featuring promotions, popular products, and links to key categories. It is designed to capture the user's attention and encourage them to explore further.
- **Product Listing Page (PLP)**: The PLP displays a list of products within a specific category or based on a search query. It typically includes filters and sorting options to help users find the products they are interested in. The PLP is designed to facilitate product discovery and encourage users to click through to individual product pages.
- **Product Detail Page (PDP)**: The PDP provides detailed information about a specific product, including images, descriptions, specifications, pricing, and customer reviews. It is designed to provide all the information a user needs to make a purchasing decision and often includes a call-to-action button to add the product to the cart or proceed to checkout.

2.1.5 The semantic web and structured data

The semantic web is an extension of the current web that aims to make data on the web more machine-readable and understandable by providing a standardized way to represent and link data using technologies such as RDF (Resource Description Framework) and OWL (Web Ontology Language). Structured data is a way of annotating web content with specific tags and formats that provide additional context and meaning to the content, making it easier for search engines and other applications to understand and process the information on the web page. One common format for structured data is JSON-LD (JavaScript Object

Notation for Linked Data), which allows webmasters to embed structured data in a web page using a JSON format that is easy for both humans and machines to read and understand [**?**].

In e-commerce websites, structured data can be used to provide detailed information about products, such as price, availability, reviews, and other attributes, which can enhance the visibility of the products in search engine results and improve the user experience by providing rich snippets in search results.

```
{
  "@context": {
    "foaf": "http://xmlns.com/foaf/0.1/",
    "title": "foaf:title",
    "name": "foaf:name",
    "homepage": {
      "@id": "foaf:workplaceHomepage",
      "@type": "@id"
    }
  },
  "@id": "http://me.markus-lanthaler.com",
  "@type": "foaf:Person",
  "title": [
    {"@value":"Dipl.Ing.", "@language":"de"},
    {"@value":"MSc", "@language": "en"}
  ],
  "name": "Markus Lanthaler",
  "homepage": "http://www.tugraz.at/"
}
```

Figure 2.2: Example of a JSON-LD document [**?**]

2.2 Crawling the Web

Web crawling is the process of systematically browsing the web and extracting content from its pages. It is a fundamental technique used in various applications, most notably in search engines, where it is used to index the web and make it searchable. Another common use of web crawling is in data mining and web scraping, where it is used to collect data from websites for analysis or other purposes. More recently, web crawling has also been used in the context of training large language models, where it is used to gather vast amounts of text data from the web to train these models. Web crawlers, also known as spiders or bots, are automated programs that perform this task.

Crawlers typically traverse the web by following hyperlinks from one page to another, starting from a set of seed URLs [**?**]. They can be designed to crawl the entire web or to focus on specific domains, topics, or types of content.

A standard crawler operates in several stages [**?**]:

1. **Seed URLs**: The crawler starts with a list of initial URLs, known as seed URLs, which are the

starting points for the crawling process.

2. **Downloading**: The crawler sends HTTP requests to the URLs and downloads the content of the web pages. The downloaded content is stored in local storage or cloud storage for further processing.

3. **Extracting links**: The crawler parses the downloaded pages to extract hyperlinks, which are then added to a crawl frontier.

A crawl frontier is a data structure that manages the list of URLs to be crawled, ensuring that the crawler does not revisit the same URL multiple times and that it adheres to any specified crawling policies (e.g., depth limits, domain restrictions). The crawl frontier can also be used as a queue to manage iterative and repeated crawling, allowing the crawler to revisit pages at regular intervals to check for updates, maintaining the freshness of the dataset [**?**].

Traditional web crawlers worked by sending HTTP requests to web servers and downloading the HTML content of web pages. However, such crawlers may struggle with modern websites that heavily rely on JavaScript and AJAX to render content dynamically via client-side rendering [**?**]. Modern web crawlers often need to use headless browsers or tools like Selenium, Puppeteer, or Playwright to handle JavaScript content and ensure that they can access the full content of web pages [**?**]. These tools allow the crawler to execute JavaScript and render the page as a regular browser would, while consuming fewer resources than a full browser, enabling the crawler to access dynamic content effectively.

It is also important to consider the legal and ethical implications of web crawling.

1. **Webmaster intent**: Webmasters may not want their content to be crawled or indexed, and they can use the `robots.txt` file to specify which parts of their site should not be accessed by crawlers [**?**].

2. **Server load**: Crawling can put a significant load on web servers, especially if the crawler is not designed to be respectful of the site's resources. It is important for crawlers to implement politeness policies, such as rate limiting, to avoid overwhelming servers [**?**].

3. **Data privacy**: Crawling and scraping can raise privacy concerns, especially if personal data is collected without consent. It is crucial to ensure that any data collected through crawling is handled in compliance with relevant data protection laws and regulations, such as the General Data Protection Regulation (GDPR) in the European Union [**?**].

A significant challenge in modern web crawling is the use of anti-bot measures by websites, such as CAPTCHAs, IP blocking, and rate limiting, which can hinder the crawling process [**?**]. Advanced security services such as Cloudflare employ fingerprinting techniques to identify and block automated traffic, making it increasingly difficult for crawlers to access content on certain websites [**?**]. Crawlers need to be designed to navigate these challenges while still adhering to ethical and legal guidelines Moreover, content hidden behind paywalls or login screens can also pose challenges for crawlers, as they may require authentication or subscription to access the content. Most crawlers, including Google's [**?**] generally do not access such content, unless explicit permission and credentials are provided by the site owner.

2.2.1 Summary

Web crawling involves a range of technical and ethical considerations. Static crawlers are simple and efficient but fail on JavaScript-heavy pages, while headless browser-based crawlers handle dynamic content and client-side rendering at highest resource costs. Anti-bot measures and legal/ethical constraints add further complexity that must be carefully managed in any crawling system. Table 2.1 summarises the key crawler types and their trade-offs.

Crawler Type	Strengths	Limitations
Static HTTP crawler	Lightweight, fast, low resource usage	Cannot handle JavaScript / client-side rendering
Headless browser	Handles dynamic content; renders JS fully	High resource usage; slower at scale

Table 2.1: Summary of web crawler types and their trade-offs

2.3 Boilerplate detection and removal

A web page is a document that contains both content and presentation information. The content of a web page is the information that is relevant to the user, while the presentation information is the information that is used to display the web page. Boilerplate refers to the presentation information that is not relevant to the user, such as navigation menus, advertisements, and other non-content elements.

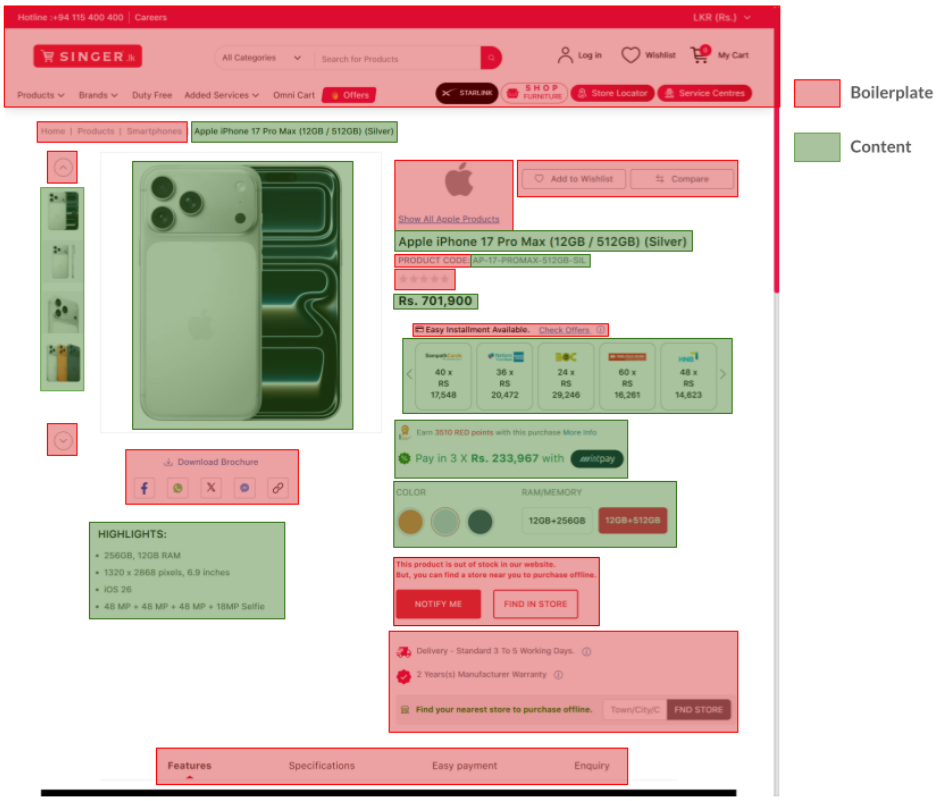

Figure 2.3: Boilerplate vs. content on a web page

Boilerplate detection and removal is the process of identifying and removing boilerplate from a web page so that only the relevant content remains. This is an important task for many applications, such as web

scraping, information retrieval, and natural language processing as boilerplate/template content can interfere with the accuracy of these applications.

There are primarily two approaches to boilerplate detection and removal [**?**]:

1. **Site-Level Methods**: These methods analyze multiple pages from the same website to identify common patterns and structures that are likely to be boilerplate. By comparing the structure and content of different pages, site-level methods can effectively identify and remove boilerplate elements that are consistent across the site.
2. **Page-Level Methods**: These methods analyze each page individually to identify boilerplate elements based on features such as link density, text density, and the structure of the HTML document.

2.3.1 Site-level methods

Site Style Trees (SST)

Using the DOM from a sample of pages from a website, a site style tree (SST) can be constructed to represent the common structure of that specific website. For each unique subtree which appears in the SST, a count is maintained to track how many pages contain that subtree. Subtrees that appear in a large number of pages are likely to be boilerplate, while those that appear in fewer pages are more likely to be content. By analyzing the SST, it is possible to identify and remove boilerplate elements from the pages of the website, leaving only the relevant content. [**?**]

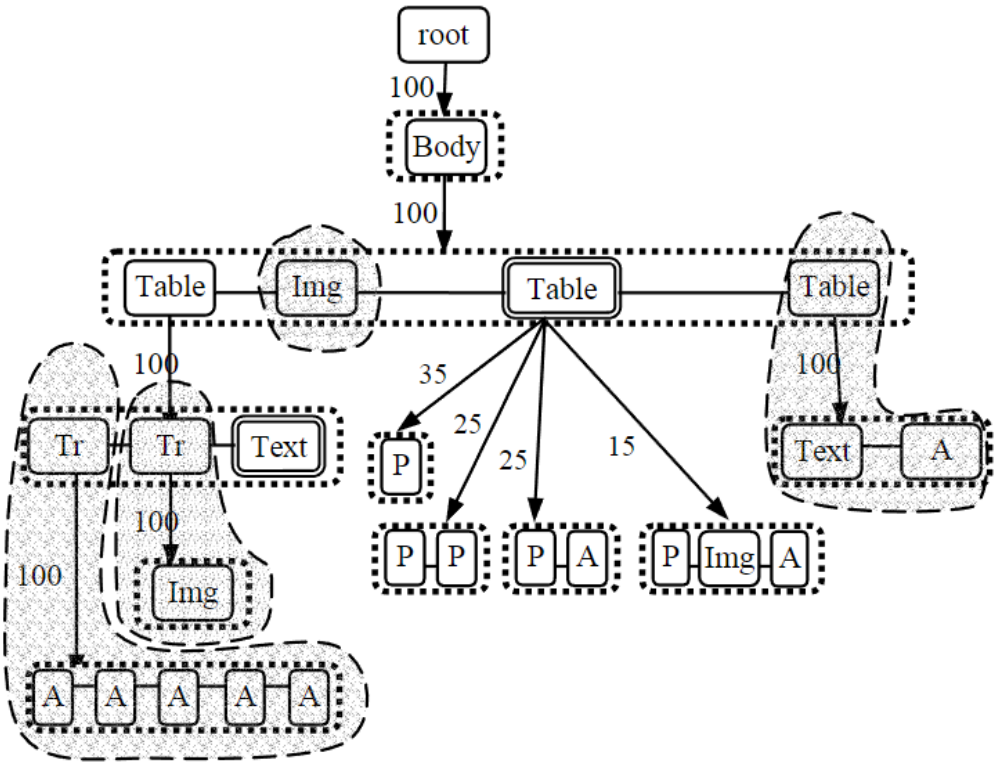

Figure 2.4: An example of site style trees (SST) [**?**]

Using subtree hashes

Similar to site style trees, subtree hashes can be used to identify boilerplate elements across multiple pages of a website. By computing a hash for each subtree in the DOM of a page, it is possible to compare these hashes across different pages to identify common subtrees that are likely to be boilerplate. Subtrees that have the same hash across multiple pages are likely to be boilerplate, while those that have unique hashes are more likely to be content. This method can be computationally efficient and effective in identifying boilerplate elements. [**?**]

2.3.2 Page-level methods

Body Text Extraction (BTE)

This is a heuristic-based method based on the observation that the main content of a web page usually contains long streams of uninterrupted text with a low HTML tag density. By analyzing the structure of the HTML document and identifying areas with high text density and low tag density, it is possible to extract the main content of the page. While this is a simple method, one major drawback is that modern web pages are extremely complex and may not always follow this pattern, leading to inaccurate results. [**?**]

Shallow text features

This method first chunks the web page into smaller sections based on the structure of the HTML document. It then computes various features for each chunk, such as link density (the ratio of links to text), text density (the ratio of text to HTML tags), and other structural features. High link density is generally indicative of boilerplate, while high text density is more indicative of content. [**?**]

This also builds on the concept of functional short text vs descriptive long text, where functional short text, characterized by short, incomplete sentences (e.g., home, contact us) is more likely to be boilerplate, while descriptive long text, characterized by full sentences and higher complexity is more likely to be content. [**?**]

Supervised machine learning techniques

While BTE and shallow text features can be effective in some cases, they may not always be sufficient to accurately identify boilerplate elements, especially on modern web pages with complex structures. Supervised machine learning techniques, such as neural networks and multi-layer perceptrons (MLPs), can be trained on labeled datasets of web pages to learn more complex patterns and features that distinguish boilerplate from content. By using a large and diverse training dataset, these models can learn to generalize well to new web pages and can be more effective in accurately identifying boilerplate elements. These techniques generally show a significant improvement in performance compared to heuristic-based methods, especially on modern web pages with complex structures. [**?**]

Two examples of supervised machine learning techniques for boilerplate detection and removal are Web2Text [**?**], which uses Convolutional Neural Networks (CNNs) to predict probabilities for chunks of a page, and the transition between blocks, and BoilerNet [**?**], which use bidirectional LSTMs that learn dependency between blocks and their neighbours.

2.3.3 Performance improvements

Removing boilerplate content from web pages have shown to significant performance gains in downstream tasks such as web page classification and clustering, as it allows these tasks to focus on the relevant content of the page rather than being distracted by irrelevant boilerplate elements. In addition, it also reduces the amount of data that needs to be processed, which can lead to faster processing times and improved efficiency in these tasks.

Using site style trees (SSTs) to remove boilerplate has been shown to improve the accuracy in web page classification tasks from 0.625 to 0.954 [**?**]. Using the same method, the average F1-score for k-means clustering also improved from 0.506 to 0.751.

2.3.4 Summary

Boilerplate detection and removal is a problem with a number of possible approaches ranging from simple rule-based models to deep learning models. Site-level methods are effective when multiple pages from the same website are available, while page-level methods operate on individual pages without any prior site knowledge. Supervised machine learning techniques offer the highest accuracy but require labeled training data. Table 2.2 summarises the key approaches reviewed in this section.

Method	**Type**	**Strengths**	**Limitations**
Site Style Trees (SST) [**?**]	Site-level	Cross-page detection, improves classification & clustering	Needs multiple pages from same site
Subtree hashes [**?**]	Site-level	Efficient, easy to implement	Needs multiple pages, fragile to DOM changes
Body Text Extraction (BTE) [**?**]	Page-level	No training data, lightweight	Poor accuracy on complex pages
Shallow text features [**?**]	Page-level	Interpretable, no training data	Hand-crafted heuristics, limited generalisation
Web2Text [**?**]	Page-level	Learns complex patterns	Requires labeled data
BoilerNet [**?**]	Page-level	Models block dependencies, best accuracy	Requires labeled data, higher complexity

Table 2.2: Summary of boilerplate detection and removal approaches

2.4 Web page classification

Web page classification is the process of categorizing web pages in one or more predefined classes based on their content, structure, and visual properties. This can be considered to be a pre-processing task of information extraction/retrieval, as the method and types of information extraction/retrieval can be different for different types of web pages. For example, the method of information extraction for a e-commerce website's product page may be different from the method of information extraction for it's contact details page. Web page classification, while fundamentally can be considered to be a text document classification task, is a unique task due to the complex structure of web pages and the presence of both content and presentation information.

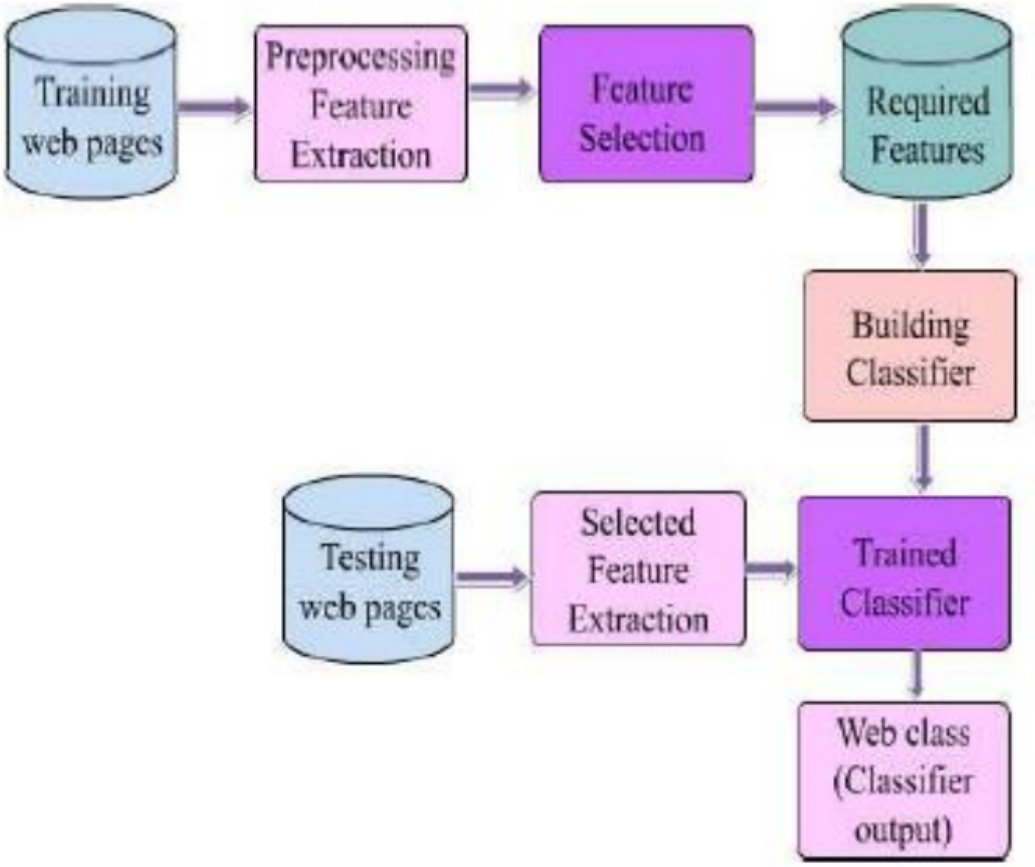

Figure 2.5: General process for web page classification [**?**]

2.4.1 Web page classification techniques

Web page classification techniques can be broadly categorized into three categories:

1. Keyword-based techniques: These techniques rely on the presence of specific keywords or phrases in the web page and its frequency to classify it into a particular category.
2. Supervised learning techniques: These techniques use labeled training data to train a machine learning model to classify web pages into different categories based on their content and structure.
3. LLM based techniques: These techniques utilize large language models (LLMs) to classify web pages.

Keyword-based techniques

Keyword-based techniques are fairly simple, rule based approaches which do not require a labeled training or training. Documents are assigned to categories based on the highest match count of specific keywords, or on keyword density (the ratio of the number of occurrences of a keyword to the total number of words in the document). While such techniques are simple and easy to implement, they can be inaccurate and perform poorly on more complex pages [**?**]. More advanced techniques such as Term Frequency - Inverse Document Frequency (TF-IDF) can be used to improve the performance of keyword-based techniques by giving more weight to keywords that are more relevant to a particular category and less weight to common keywords that appear in many categories. Combining TF-IDF with traditional machine learning classifiers have shows to have strong performance in web page classification tasks [**?**].

Supervised learning techniques

Supervised learning techniques for web page classification involve training a machine learning model on a labeled dataset of web pages, where each web page is associated with a specific category or class. The model learns to classify web pages based on the features extracted from the content and structure of the web pages. Classical algorithms such as Naive Bayes, Support Vector Machines (SVM), k-Nearest Neighbour (kNN) and Decision Trees have been used for web page classification, while SVMs have been shown to be particularly effective for this task [**?**].

More recently, deep learning techniques utilizing Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have been applied to web page classification tasks, showing improved performance over traditional machine learning algorithms. Transformer-based models such as BERT (Bidirectional Encoder Representations from Transformers) have also been applied to web page classification tasks, showing strong performance due to their ability to capture contextual information in the text. However, more simple models usually offer a better efficiency-performance trade-off, and are often preferred in settings where computational resources may be limited [**?**].

LLM based techniques

Large language models (LLMs) have shown to be effective in various natural language processing tasks, including web page classification. While this is a very complex technique, the existence of foundational models by OpenAI, Google, Anthropic and others, has made it more accessible and easier to implement via an API call. A significant benefit of using LLMs, is that it can be used for zero-shot classification, which means it can be used when there are no training example available for the specific classification task. Another advantage is that it can handle very long documents (contexts) with high accuracy. A major drawback though, is the high cost of this method as API calls to LLMs can be expensive, especially for long, feature rich documents such as web pages. Moreover, the per-page inference time can also be significantly higher compared to traditional machine learning techniques, which can be a concern in settings where real-time classification is required or when processing large volumes of web pages [**?**]

2.4.2 Techniques without negative examples

As web page labelling is generally a manual and time-consuming process, in some cases, it may be difficult to have a complete labeled dataset consisting of both positive and negative classes (e.g. product pages vs. non-product pages). It can be easier to just have a set of positive examples (e.g. product pages) without having a complete set of negative examples (e.g. non-product pages). In such cases, the Positive Example Based Learning (PEBL) framework [**?**] can be to identify strong negatives from the unlabeled data and then train a classifier using the positive examples and the identified strong negatives. This approach can be effective in situations where it is difficult to obtain a complete labeled dataset.

2.4.3 Effect of noise removal

In the context of web page classification, noise refers to the irrelevant information on a web page that can interfere with the accuracy of the classification process. This can include boilerplate content, advertisements, navigation menus, and other non-content elements. The presence of noise can lead to misclassification of

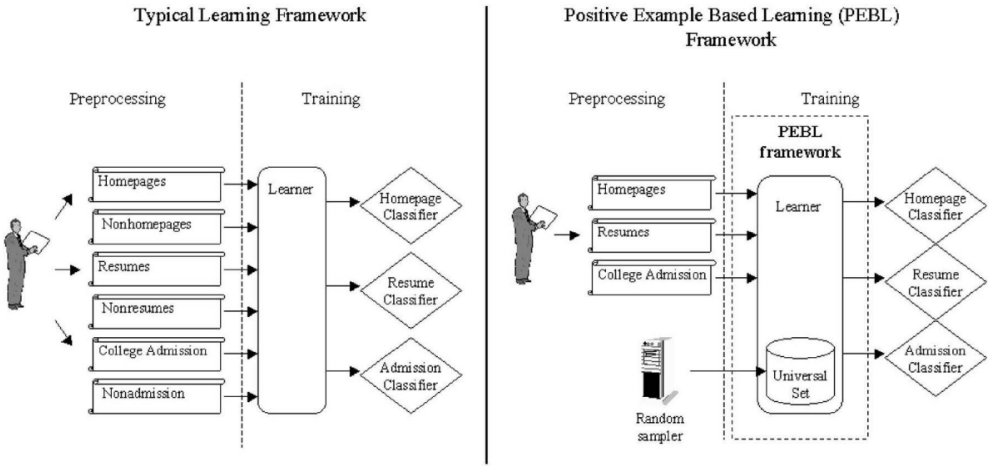

Figure 2.6: Typical framework vs. using PEBL [**?**]

web pages, as the classifier may be influenced by the irrelevant information rather than the relevant content. By removing noise from web pages, the classifier can focus on the relevant content, which can lead to more accurate classification results. In [**?**], it was indicated that performance of web page classification improved by 10% to 15% after noise removal.

2.4.4 MarkupLM

MarkupLM [**?**] is a pre-trained transformer based model designed for markup-language based documents such as HTML and XML. Unlike standard techniques, which treat web pages as a block of text, MarkupLM takes into account the structure of the web page by incorporating the HTML tags and their relationships into the model. It introduces XPath embeddings, which capture for each node, it's position in the DOM tree, its relationship with other nodes, and the HTML tag information. This also allows MarkupLM to capture the hierarchical structure of web pages, which can be important for understanding the context and meaning of the content on the page. This has resulted in significant performance improvements compared to text-only based techniques and models.

2.4.5 E-commerce web page classification

E-commerce web page classification is a specific application of web page classification that focuses on categorizing web pages related to e-commerce, such as product pages, product list pages, blog pages etc. This can be particularly challenging as e-commerce sites contain complex and repetitive structures, which can also vary significantly across different sites. In addition, the presence of noise such as advertisements, navigation menus, and other non-content elements can also hinder the classifier's performance. In [**?**], it was shown that Multi-Layer Perceptron (MLP) based models and SVMs can achieve a strong performance on e-commerce web page classification tasks.

2.4.6 Summary

Web page classification techniques vary in their complexity, data requirements, and resource requirements. Keyword-based methods are simple but inaccurate on complex pages, while supervised learning and transformer-

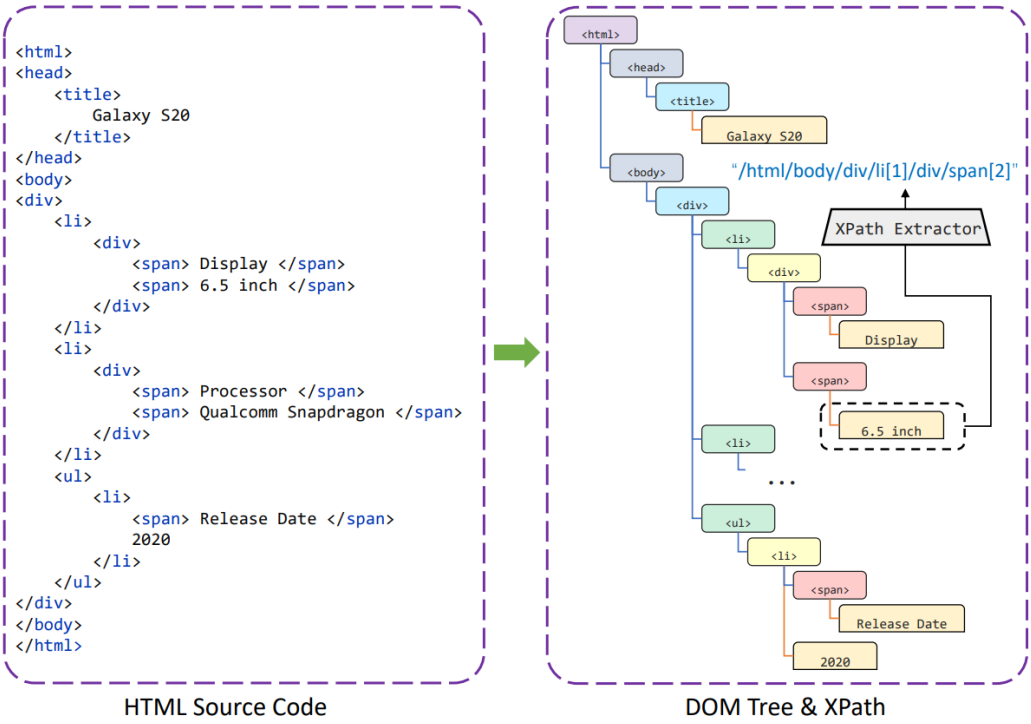

Figure 2.7: DOM tree and XPath in HTML documents [**?**]

based models offer stronger performance while requiring extensive labelled training data and compute resources. LLMs provide a zero-shot alternative but are expensive at scale. Table 2.3 summarises the key approaches reviewed in this section.

Technique	**Strengths**	**Limitations**
Keyword-based (TF-IDF)	Simple, no training data, fast	Low accuracy on complex pages
Classical ML (SVM, kNN, Naive Bayes)	Well-studied, strong baseline, efficient	Requires labeled data, limited context understanding
Deep learning (CNN, RNN)	Captures sequential patterns, better generalisation	High data and compute requirements
Transformer-based (BERT, MarkupLM)	Captures context, leverages HTML structure	Computationally expensive, needs fine-tuning
LLM-based	Zero-shot, handles long context	High API cost, slow inference at scale
PEBL (positive examples only)	No negative examples needed	Less accurate than fully supervised methods

Table 2.3: Summary of web page classification techniques

2.5 Web information extraction

Web information extraction is the process of extracting structured data from unstructured web content. While traditionally, this was done using rule-based approaches, where developers would write specific rules to identify HTML elements via their XPath or CSS selectors, modern approaches have shifted towards machine learning and deep learning techniques. These methods leverage the underlying structure of the webpage, such as the Document Object Model (DOM), to identify and extract relevant information. For

instance, models can be trained to recognize patterns in the DOM tree, allowing them to extract data even when the webpage layout changes, which is a common occurrence on the web. Another requirement is for zero-shot extraction, where the model can extract information from webpages it has never seen before, without needing to be retrained for each new website. In the context of e-commerce, web information extraction involves tasks such as extracting product details (e.g., name, price, description) from product pages, which can be challenging due to the diversity of webpage designs and the presence of boilerplate content.

2.5.1 Embedded semantic metadata

More modern websites, especially those build on top of popular frameworks and content management systems, often include embedded semantic metadata in their HTML code to provide additional context about the content of the page. This metadata is used to provide search engine crawlers and other applications with structured information about the content of the page, which can be used to improve search results and enable rich snippets in search engine results pages (SERPs).

Microdata, RDFa, and JSON-LD are common formats for embedding semantic metadata in HTML.

- **Microdata** [?] is a specification for embedding structured data in HTML documents using a simple syntax. It allows developers to annotate their HTML with additional attributes that provide information about the content of the page.
- **RDFa** [?] (Resource Description Framework in Attributes) is a W3C recommendation that provides a way to embed RDF (Resource Description Framework) data in HTML documents using attributes. It allows for more complex and flexible annotations compared to Microdata.
- **JSON-LD** [?] (JavaScript Object Notation for Linked Data) is a lightweight format for encoding linked data using JSON. It is often used to embed structured data in web pages, and it can be easily parsed by search engines and other applications.

Most of these metadata formats are based on the Schema.org [?] vocabulary, which provides a standardized set of types and properties for describing various entities and their relationships on the web. For example, a product page on an e-commerce website might include JSON-LD markup that describes the product's name, price, description, and availability, as shown in Listing 2.1.

```
{
  "@context": "https://schema.org/",
  "@type": "Product",
  "@id": "https://lifemobile.lk/product/honor-magic-v5-5g-16gb-ram-512gb/#product",
  "name": "Honor Magic V5 5G 16GB RAM 512GB",
  "url": "https://lifemobile.lk/product/honor-magic-v5-5g-16gb-ram-512gb/",
  "description": "7.95 inches Display, 5820mAh silicon-carbon battery, Snapdragon 8 Elite,
      1 Year Company Warranty",
  "image": "https://lifemobile.lk/wp-content/uploads/2025/11/Honor-Magic-V5-5G-16GB-RAM
     -512GB.jpg",
  "sku": "honor-magic-v5-5g-16gb-ram-512gb",
  "offers": [
    {
      "@type": "Offer",
```

```
      "priceSpecification": [
        {
          "@type": "UnitPriceSpecification",
          "price": "634285.71",
          "priceCurrency": "LKR",
          "valueAddedTaxIncluded": false,
          "validThrough": "2027-12-31"
        }
      ],
      "priceValidUntil": "2027-12-31",
      "availability": "http://schema.org/InStock",
      "url": "https://lifemobile.lk/product/honor-magic-v5-5g-16gb-ram-512gb/",
      "seller": {
        "@type": "Organization",
        "name": "Life Mobile",
        "url": "https://lifemobile.lk"
      }
    }
  ]
}
```

Listing 2.1: Example JSON-LD product markup from an e-commerce webpage with the Schema.org vocabulary

Of the three most common metadata formats, JSON-LD is the most widely used for embedding structured data in web pages, particularly for e-commerce websites. An analysis of the 100 most popular e-commerce websites in the US and UK in 2023 found that only 45% of URLs did not contain JSON-LD metadata, and 27% contained unstructured JSON-LD metadata with errors. [**?**]

Therefore, while embedded semantic metadata can be a valuable source of structured data for web information extraction, it is not universally present or consistently implemented across all websites, which can limit its effectiveness as a sole method for extracting information from the web.

2.5.2 Vision-based approaches

While the Document Object Model (DOM) provides a structured representation of a webpage, it can be complex and noisy, and usually fails to reflect the actual semantic grouping of content as perceived by human users. Vision-based approaches to web information extraction address this issue by treating the webpage as an image and leveraging computer vision techniques to identify and extract relevant information. The **Vision-based Page Segmentation (VIPS) algorithm** [**?**] was developed to segment a webpage into visually coherent blocks, which can then be analyzed to extract structured data.

The VIPS algorithm works by building a hierarchical tree structure of the webpage based on its visual layout, rather than its DOM structure. Each node in the tree represents a visually coherent block of content, and the algorithm uses various visual cues such as font size, color, and spatial relationships to determine how to segment the page.

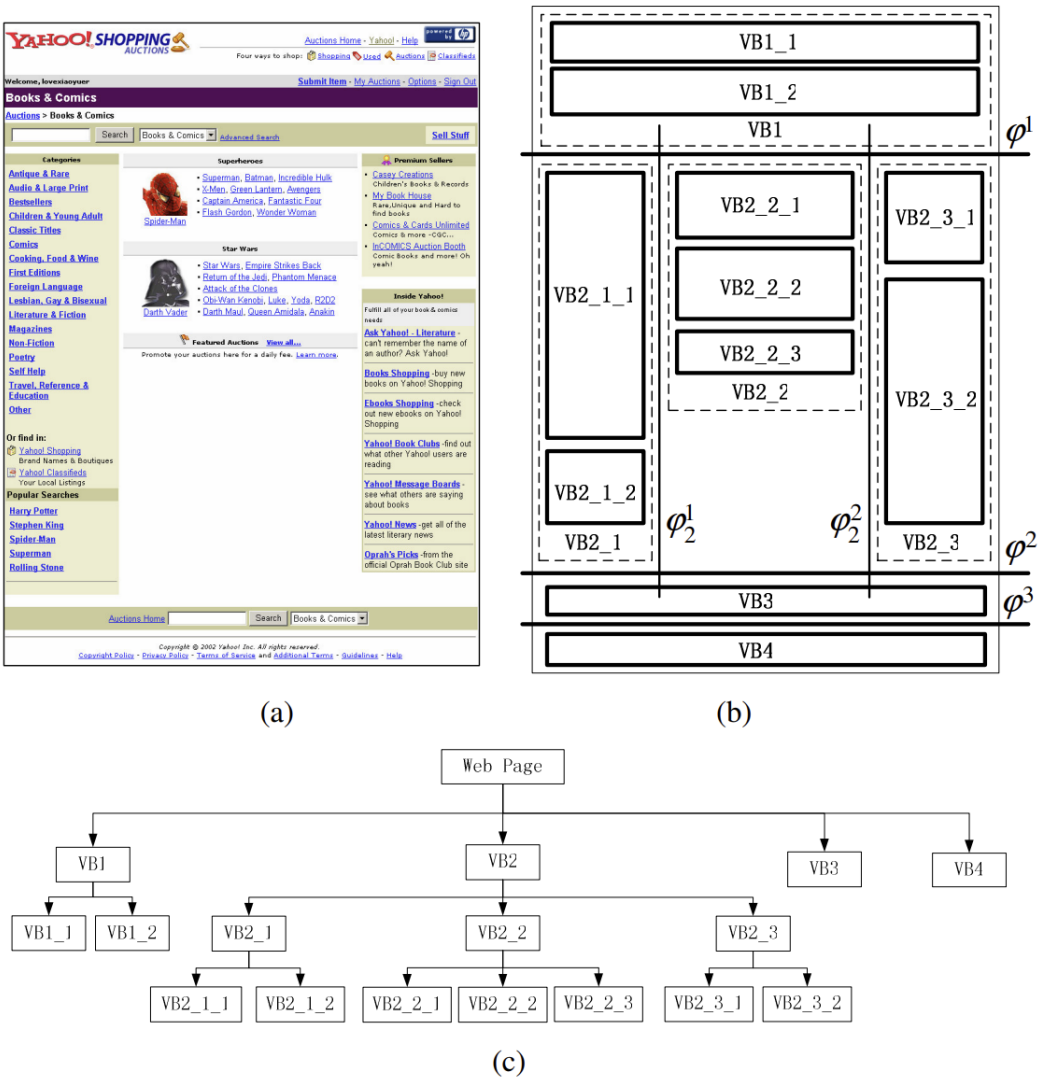

Figure 2.8: Image segmentation of a webpage using the VIPS algorithm [?]

2.5.3 DOM tree node classification

Another approach to web information extraction is to treat the problem as a classification task, where each node in the DOM tree is classified as either relevant or irrelevant for the information extraction task at hand. This approach can be implemented using machine learning or deep learning models, which can learn to identify relevant nodes based on features such as the node's tag name, attributes, text content, and its position in the DOM tree. Graph Neural Networks (GNNs) have been used to model the relationships between nodes in the DOM tree, allowing for more effective classification of relevant nodes.

ZeroShotCeres [?] is a model that uses a GNN to classify nodes in the DOM tree for web information extraction. It is designed to perform zero-shot extraction, meaning it can classify nodes on webpages it has never seen before, without needing to be retrained for each new website. The model is trained on a large dataset of webpages with labeled nodes, and it learns to generalize from this training data to classify nodes on new webpages based on their features and their relationships to other nodes in the DOM tree.

AttriSage [?] uses a similar approach to extract product attributes from e-commerce webpages by classifying nodes by treating the task as a multi-label node classification problem, where each node can be classified with multiple labels corresponding to different product attributes (e.g., name, price, description). It uses a GNN

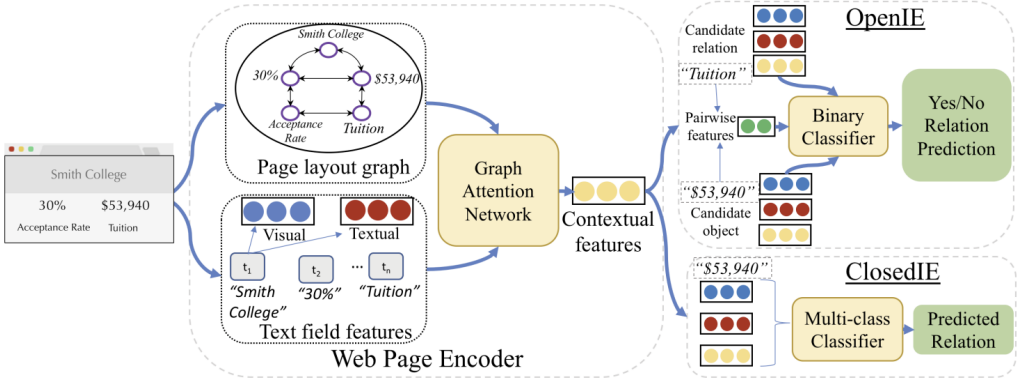

Figure 2.9: Node classification using ZeroShotCeres [**?**]

to model the relationships between nodes in the DOM tree, allowing it to effectively classify nodes based on their features and their relationships to other nodes, even when the webpage layout varies significantly across different e-commerce websites.

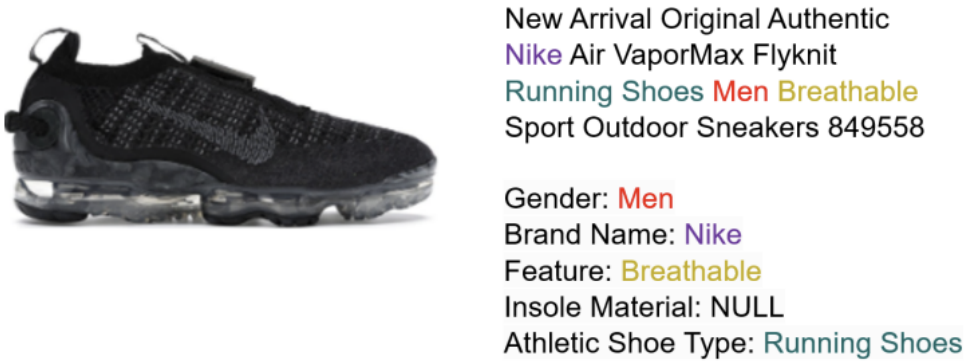

Figure 2.10: Product attribute extraction [**?**]

2.5.4 Markup and layout-based approaches

Recent transformer architectures have been developed that integrate the structural hierarchy of HTML directly into language representations, allowing for more effective web information extraction by leveraging both the textual content and the structural layout of the webpage. Unlike traditional models which only encode the textual content of a node, these models also encode the position of the node in the DOM tree, which can provide valuable context for understanding the relationships between different elements on the page.

MarkupLM [**?**] encodes both the textual content of a node and its position in the DOM tree using a specialized XPath embedding layer. It is a pretrained language model that can be fine-tuned for various web information extraction tasks. MarkupLM also treats information extraction as a token classification task, where each token in the input sequence is classified as either relevant or irrelevant for the extraction task, allowing it to effectively identify and extract relevant information from the webpage. Since MarkupLM does not actually render the webpage, it is very lightweight and can be used for large-scale web information extraction without the overhead of rendering webpages, while still leveraging the structural information

provided by the DOM tree.

MarkupLM uses BERT architecture and is pretrained on 24 million web pages from the Common Crawl [**?**] dataset, which allows it to learn a wide range of patterns and structures commonly found in web pages, making it effective for zero-shot extraction on new websites.

WebFormer [**?**] is a transformer architecture that integrates the structural hierarchy of HTML directly into language representations by encoding the position of each node in the DOM tree using a specialized embedding layer. There are three types of tokens used in WebFormer in the input layer.

- **Field tokens**represent the different fields that are being extracted from the webpage, such as product name, price, description, etc.
- **HTML tokens** represent the HTML path of each node in the DOM tree, which provides information about the structure and layout of the webpage, allowing the model to understand the relationships between different elements on the page.
- **Text tokens** represent is an embedding of the textual content of each node in the DOM tree, which provides information about the content of the webpage.

WebFormer uses four specialized attention patterns (HTML-to-HTML, HTML-to-Text, Text-to-HTML, and Text-to-Text) to allow the model to effectively leverage both the structural information provided by the HTML tokens and the content information provided by the text tokens, enabling it to perform accurate web information extraction even when the webpage layout varies significantly across different websites.

Performance comparison

Benchmark	**Metric**	**MarkupLM**	**WebFormer**
SWDE (1 seed site)	F1	82.11%	–
SWDE (5 seed sites)	F1	97.37%	92.46%
SWDE	EM	–	86.58%
SWDE (Products)	F1	–	83.37%
SWDE (Products)	EM	–	80.67%
WebSRC	F1	74.47%	–
WebSRC	EM	68.39%	–

Table 2.4: Performance comparison of MarkupLM and WebFormer on standard benchmarks

2.5.5 Multimodal approaches

Multimodal approaches to web information extraction leverage multiple sources of information, such as the textual content and the visual layout of the webpage, to improve the accuracy and robustness of information extraction.

LayoutLMv3 [**?**] is one such model that encodes text (via OCR) and layout coordinates and image patches. It is a pretrained multimodal transformer model that can be fine-tuned for various web information extraction

tasks. LayoutLMv3 uses a combination of text, layout, and image information to perform information extraction, allowing it to effectively identify and extract relevant information from the webpage even when the layout varies significantly across different websites. While LayoutLMv3 can achieve high accuracy on web information extraction tasks, it is computationally expensive due to the need to render the webpage and perform OCR to extract text, which can limit its scalability for large-scale web information extraction tasks.

2.5.6 LLM-based approaches

Large Language Models (LLMs) excel at zero-shot information extraction tasks, where the model can extract information from webpages it has never seen before without needing to be specifically trained. LLMs can be prompted with the HTML content of a webpage and a specific extraction task, and they can generate structured data based on the provided input. For example, a prompt might include the HTML content of a product page and a request to extract the product name, price, and description, and the LLM would generate a structured output containing this information.

A drawback of LLM-based approaches is that they can be computationally expensive, especially for large webpages with complex layouts, and they may not always produce accurate results, particularly when the webpage contains a lot of noise or irrelevant information. Therefore, it is essential that the number of tokens provided as input to the LLM is limited, which can be achieved through boilerplate removal techniques that filter out irrelevant content from the webpage before feeding it into the LLM, and by converting documents into Markdown format, which can help to preserve the structural information of the webpage while reducing the amount of token introduced due to encoding HTML tags [**?**].

2.5.7 Summary

Web information extraction approaches vary significantly in their reliance on structure, visual layout, and scale. Rule-based methods are precise but brittle, while machine learning and transformer-based models offer greater generalisability at the cost of increased complexity. Embedded semantic metadata provides a lightweight extraction path but is inconsistently adopted across the web.

Approach	**Strengths**	**Limitations**
Embedded metadata	Structured, low overhead	Inconsistent adoption
Vision-based	Human-like layout perception	High rendering cost
DOM node classification	Zero-shot, structural reasoning	Complex training data
Markup & layout	High accuracy, lightweight	Domain shift degradation
Multimodal	Robust on visually complex pages	Computationally expensive
LLM-based	Zero-shot, flexible	Token limits, cost, noise sensitivity

Table 2.5: Summary of web information extraction approaches

2.6 Product matching

Product matching is the task of identifying and linking different web pages which represent the same underlying product across different e-commerce websites. These kinds of tasks are also commonly known as entity resolution, record linkage, or deduplication. Brute-forcing product matching is a computationally expensive task, with a quadratic time complexity ($O(n^2)$) in the number of products [**?**].

To mitigate this issue, most pipelines employ a filtering-verification based approach, where a blocking step is used to reduce the number of candidate pairs, followed by a verification step to determine if the candidate pairs are indeed matches. The filtering step usually employs a computationally inexpensive method, with low recall, to quickly eliminate a large number of non-matching pairs, while the verification step uses a more sophisticated method, with higher recall, to accurately identify the matching pairs from the candidate set.

Moreover, as the data we have is unstructured, noisy, and often incomplete, product matching is a challenging task that requires sophisticated techniques to achieve high accuracy.

2.6.1 Blocking and filtering techniques

Blocking involves grouping documents into blocks based on certain criteria, such as shared attributes or similar values, to reduce the number of candidate pairs that need to be compared in the verification step.

Blocking Key Values (BKV)

Blocking Key Values is a traditional blocking technique that uses specific attributes (keys) of an entity to create blocks. For example, in product matching, the blocking key could be the product brand, category, or a combination of attributes. While this technique is straightforward and simple to implement, it can lead to a high number of false negatives if the blocking key is not chosen carefully, or if the data is noisy or contains typos. Another drawback of BKV is that, it requires the key attribute to be present and correctly formatted in the data, which is not always the case in real-world scenarios. [**?**]

Sorted Neighbour Algorithm (SNA)

The Sorted Neighbour Algorithm is a more sophisticated blocking technique that sorts the records based on a specific attribute and then compares only the records that are within a certain distance of each other in the sorted list. To minimize the effect of noisy data, SNA sorts the records based on a key, and moves a sliding window against it, comparing only the records that fall within the window. [**?**]

Canopy Clustering

uses two distance thresholds, T_1 and T_2 (where $0 < T_2 < T_1 < 1$): records within T_1 of a randomly selected center are added to its canopy, while those within the tighter T_2 are removed from the candidate pool entirely. This iterative process continues until the pool is exhausted, with records potentially belonging to multiple canopies due to the looser T_1 threshold. The coarse similarity measure used for grouping (commonly TF-IDF distance) is computationally inexpensive, with the effective number of comparisons reduced to approximately $O(fn^2/c)$, where f is the average canopy membership per record and c is the

total number of canopies. However, canopy clustering is sensitive to the value selected for T_1. [**?**]

Locality Sensitive Hashing (LSH)

Locality Sensitive Hashing is a technique that hashes similar items into the same buckets. LSH uses a family of hash functions that are designed to maximize the probability of collision for similar items, while minimizing the probability of collision for dissimilar items. In most cases, a vector representation (such as TF-IDF, or document embeddings) of the data is used, and the hash functions are designed to preserve the similarity between the vectors. A benefit of LSH is that it can be pre-computed and stored in a hash table, which allows for fast retrieval of candidate pairs. [**?**]

Pairwise string and text similarity measures

This technique uses various string similarity measures to identify potential candidates for a given record. These can involves **character-based metrics** such as Levenshtein distance, Jaro-Winkler distance, and Hamming distance, which measure the similarity between two strings based on their character-level differences. Alternatively, **token-based metrics** such as Jaccard similarity, TF-IDF cosine similarity, and SoftTF-IDF, which measure the similarity based on the tokens (words) in the strings. [**?**]

Comparison of blocking and filtering techniques

Technique	**Strengths**	**Limitations**
Blocking Key Values	Computationally efficient; can achieve $O(n)$ complexity with high-quality keys	Highly sensitive to noise and typos; a single character error can cause a missed match; requires labelled attributes
Sorted Neighbour Algorithm	More noise-tolerant than BKV; $O(n \log n)$ for sorting, $O(wn)$ for the sliding window	Depends on sorting key quality; requires labelled attributes
Canopy Clustering	Reduces comparisons using coarse similarity; overlapping blocks improve recall; does not require labelled attributes	Sensitive to threshold T_1; complexity is typically $O(n^2)$
Locality Sensitive Hashing	Efficient for high-dimensional vectors; enables fast ANN search; does not require labelled attributes	Approximate by nature; true matches may be missed; complex to tune
Pairwise String Similarity	Flexible; does not require labelled attributes	$O(n^2)$ if used standalone

Table 2.6: Comparison of blocking and filtering techniques

2.6.2 Verification techniques

Once a candidate set of potential matches has been generated through blocking/filtering, the next step is to verify which of these candidates are indeed matches. This step typically involves more computationally expensive techniques, as the number of candidate pairs is significantly reduced compared to the original dataset.

Approximate Nearest Neighbour (ANN) with vector databases

ANN algorithms are designed to efficiently find the nearest neighbors of a given query point in high-dimensional spaces. They essentially trade recall for speed, by allowing for approximate matches rather than exact matches. In the context of product matching, ANN can be used to compare vector representations of candidate product records (such as embeddings, or TF-IDF representation) to determine if they are likely to be matches. There are multiple ANN algorithms available.

Hierarchical Navigable Small World (HNSW) is a state-of-the-art ANN algorithm that constructs a multi-layer graph where each layer represents a different level of granularity in the search space. The algorithm navigates through the graph to find the nearest neighbors efficiently, with a time complexity of $O(\log n)$ for search and $O(n \log n)$ for construction. [**?**]

Inverted File with Flat Compression (IVF-Flat) is an ANN algorithm that partitions the vector space into a specified number of cluster using the K-means algorithm. At query time, the system identified and searches only the nearest clusters instead of the entire index. This approach significantly reduces the search space, with a time complexity of $O(n \log n)$ for index construction and $O(k \log n)$ for search, where k is the number of clusters searched. [**?**]

These algorithms can be implemented on PostgreSQL databases using extensions like pgvector [**?**] and pgvectorscale, which provides efficient indexing and querying capabilities for high-dimensional vectors. [**?**] However, using dedicated vector databases such as Pinecode, Milvus, or Qdrant can offer faster search performance and better scalability, as they are optimized for ANN search and can handle larger datasets more efficiently. They also support a wider range of ANN algorithms and provide additional features such as distributed indexing, querying, and hybrid search. [**?**] However for smaller datasets, using a PostgreSQL database with pgvector can be a cost-effective and competive solution [**?**]

Probabilistic record linkage

Probabilistic record linkage is a technique that uses statistical models to estimate the probability that two records refer to the same entity. This approach typically involves calculating a similarity score for each pair of records based on their attributes, and then using a threshold to determine if the pair is a match or not. It computes the likelihood of a match based on the agreement and disagreement of attributes, and can incorporate prior knowledge about the data to improve accuracy using Bayesian mechanics. [**?**]

In essence, it computes three probabilities for whether two records from the dataset are a match:

- m: the probability that a given attribute agrees between two records, given that they are a match.
- u: the probability that a given attribute agrees between two records, given that they are not a match.
- λ: the prior probability that any two records are a match.

For each attribute, the **Bayes Factor** K quantifies the strength of evidence for a match as the ratio of these probabilities:

$$K = \frac{m}{u} \tag{2.1}$$

A Bayes Factor of 10 means an agreement on that attribute is ten times more likely if the records are a match than if they are not. To make weights additive across attributes, the Bayes Factor is log-transformed

into a **match weight**:

$$w = \log_2\left(\frac{m}{u}\right) \tag{2.2}$$

A positive weight provides evidence for a match, while a negative weight provides evidence against it. The total match weight $W = \sum_i w_i$ across all attributes is then converted into a final **match probability** via:

$$P(\text{match}) = \frac{\lambda \cdot 2^W}{1 - \lambda + \lambda \cdot 2^W} \tag{2.3}$$

For example, with a uniform prior $\lambda = 0.5$, a total weight of $W = 3$ corresponds to a match probability of approximately 90%, while $W = 7$ yields approximately 99%. [**?**]

A drawback of using probabilistic record linkage is that it requires attribute-value pairs to be present and correctly formatted in the data, which is not always the case in real-world scenarios. Additionally, it can be computationally expensive to calculate the similarity scores and probabilities for large datasets, especially if the number of attributes is large.

Deep learning and pre-trained language models

Deep learning techniques, particularly those leveraging pre-trained language models (PLMs) such as BERT, have shown significant promise in entity resolution tasks, including product matching. These models can capture complex semantic relationships between product attributes and descriptions, making them particularly effective in handling noisy and unstructured data. By fine-tuning PLMs on labeled datasets of product pairs, the models can learn to generate embeddings that reflect the similarity between products, which can then be used for both blocking and verification steps.

DeepMatcher [**?**] is a framework which introduced a deep learning-based approach to entity resolution, using customized Recurrent Neural Networks (RNNs) and attention mechanisms to learn representations of product pairs and predict matches. It aggregates attribute values into vector representations before performing pairwise comparisons. While DeepMatcher demonstrated good performance on unstructured data, it can struggle with long sequences of text.

The DITTO framework [**?**] treats entity resolution and a sequence-to-sequence binary classification task. It serializes the attribute-value pairs of two records into a single string, which is then passed through a transformer model such as BERT or RoBERTa to preict whether the two records match. This technique has shown stronger performance than DeepMatcher, however it requires attribute-value pairs to be present in the data.

RexBERT [**?**] is a context-specific model based on BERT which has been fine-tuned on an e-commerce product dataset. While general purpose pre-trained language models can capture semantic relationships, they may not be optimized for the specific language and structure of product data. RexBERT is designed to better capture the nuances of product descriptions and attributes, which can lead to improved performance in product matching tasks.

Multi-modal approaches

Multi-modal approaches involve using both textual and visual information to perform product matching. For example, a product may have a textual description as well as an image, and both of these can be used to determine if two products are a match. By combining features from both modalities, multi-modal

approaches can often achieve higher accuracy than methods that rely solely on text or images. For instance, a product may have a similar description to another product, but the images may reveal that they are different products (e.g., different colors or designs). Conversely, two products may have different descriptions but similar images, which could indicate that they are the same product with different attribute values (e.g., different sizes or materials). While this technique is significantly more complex and resource intensive than text-only approaches, its effectiveness in e-commerce datasets may not be as expected, as different product variations can have very similar images, and product descriptions can often contain more discriminative information than images alone. [**?**]

Comparison of verification techniques

Technique	**Strengths**	**Limitations**	**Requires Attr. Data?**
HNSW	Graph-based; $O(\log n)$ search; high recall, low latency	Memory-intensive; slow index build	No
IVFFlat	Faster builds and lower memory than HNSW; suits frequent re-indexing	Lower recall; depends on cluster count	No
Probabilistic Record Linkage	Weighs multiple fields; interpretable; robust on messy data	Requires tuning of m, u, and thresholds	Yes
DeepMatcher	Semantic similarity via RNNs and attention; strong on dirty data	Slow training; OOM errors on large datasets	No
DITTO	Strong performance; label-efficient	512-token limit; high GPU cost	Yes
RexBERT	E-commerce optimised; 8,192-token context; efficient vs. larger models	Domain-specific; needs large retail corpus	No
Multi-modal	Fuses text and image; effective in visual domains	High compute cost; weak image signal in some domains	No

Table 2.7: Comparison of verification techniques for product matching

2.7 Related work

2.7.1 Consumer-facing public systems

Several publicly accessible systems exist which aggregate product information from multiple online retailer, and make it available to consumers. They offer different features and the underlying techniques used to extract and process the data vary widely.

Some of the most popular systems include:

- **CamelCamelCamel**[1] tracks over 18 million products on Amazon, generating over 72 million price

[1] `https://camelcamelcamel.com/`

records per day. While it primarily uses the Amazon Product Advertising API to collect product information, it also employs distributed headless web scraping to extract information when API rate-limits are reached, and also to extract unexposed metrics such as Buy Box variations. A major drawback of CamelCamelCamel is that it is limited to the Amazon ecosystem. [**?**]

- **Keepa**[2] monitors over 5.6 billion products over 11 global marketplaces. It provides detailed charts including sales rank, Buy Box statistics, and inventory levels, updated as frequently as every 15 minutes. It provides deeper metrics such as sales rank history, inventory leve, and Buy Box statistics. While its free-tier offers basic features, the more advanced features require a premium subscription. [**?**]

- **PayPal Honey**[3] operates across thousands of retailers. Its droplist feature allows users to track price changes for specific products, and it also provides a browser extension that automatically applies coupon codes at checkout. Honey collects product information through a combination of web scraping and partnerships with retailers, and it also uses crowdsourced data from its user base to enhance its product database. [**?**]

- **37left.lk**[4] is a Sri Lankan price comparison website that allows users to compare prices of products across multiple online retailers and check historical price records. It collects product information by continuously crawling the websites of participating retailers. It also supports features such as price alerts, allowing users to receive notifications when the price of a tracked product drops below a specified threshold. Currently, 12 Sri Lankan retailers are supported. [**?**]

2.7.2 Self-hosted and open-source systems

Several self-hosted and open-source systems exist that allow users to track product prices and availability across multiple online retailers. These systems often provide more flexibility and control over the data collection and processing methods, but may require more technical expertise to set up and maintain. A key benefit that self-hosted systems offer is that they allow users to keep control of their data and privacy.

- **PriceGhost** allows price tracking from virtually any e-commerce website. Its primary highlight, is its Price Voting Modal, which runs four independent extraction methods in parallel to extract product information from a given product page. [**?**]
  - JSON-LD: Reads structured data embedded in the product page's HTML, which is often used by e-commerce websites to provide product information in a machine-readable format.
  - Site-specific scrapers: Using custom rule-based scrapers for major retailers such as Amazon, Walmart, and Best Buy.
  - Generic CSS: Using intelligent CSS selectors to extract product information from the page's HTML.
  - AI Analysis: Using LLMs to analyze the product page and extract relevant information based on the page's content and structure.

  The results from these four methods, showing each candidate with the method used and the confidence score, are presented to the user for verification and selection.

[2] `https://keepa.com/`
[3] `https://www.joinhoney.com/`
[4] `https://37left.lk/`

- **PriceBuddy**, while it also supports price tracking for major retailers out of the box, it also allows users to create custom scrapers for any e-commerce website by providing CSS selectors, regular expressions, or JSON paths. It also offers a multi-store comparison feature, which allows users to add multiple URLs for a single product entry and compare prices across different retailers. A distinct feature of PriceBuddy is its unit pricing feature, which allows users to compare prices based on the quantity or size of the product, making it easier to identify the best value per unit. It also provides support for in-stock tracking, including opt-in back-in-stock alerts. [**?**]

2.7.3 Enterprise competitive intelligence systems

Several enterprise-level competitive intelligence systems exist that provide businesses with insights into their competitors' pricing strategies, product offerings, and market trends. These systems often offer advanced features such as automated data collection, analytics dashboards, and integration with other business intelligence tools. They are typically used by larger organizations with dedicated teams for market research and competitive analysis

- **Omnia Retail**[5] provide intelligence platforms combining dynamic repricing with agentic AI. Omnia Agent analyzes pricing data and surfaces relevant insights without requiring manually curated dashboards, offering recommendations. These systems help organizations spot pricing shifts in real-time and execute pricing strategies across multiple markets. [**?**]
- **Price2Spy**[6] specializes in Minimum Advertised Price (MAP) monitoring and enforcement. It helps brands maintain pricing integrity across global reseller networks through automated evidence collection and real-time monitoring of reseller compliance. [**?**]
- **Competera**[7] is an AI-native platform combining elasticity-based pricing with predictive algorithms. It suggests optimal pricing adjustments based on customer behavior and market conditions, reducing operational costs by up to 30% while improving pricing accuracy and customer lifetime value. [**?**] The platform excels in product matching and relationship management through several capabilities:
  - *Product Relationship Management*: Establishes linear or hierarchical dependencies between products, allowing synchronized pricing adjustments across related items.
  - *Dynamic Product Assignment*: Automatically distributes products to pricing campaigns based on specific sales parameters and business constraints.

[5] `https://www.omniaretail.com`
[6] `https://www.price2spy.com`
[7] `https://www.competera.net`

2.7.4 Summary of Related Systems

System	Type	Key Features	Primary Techniques Used
CamelCamelCamel	Public (Free)	Amazon-exclusive price tracking, historical price charts, and email price-drop alerts	Web scraping and official Amazon Product Advertising API
Keepa	Public (Freemium)	Detailed Amazon price/stock history, sales rank tracking, Buy Box statistics, and inventory monitoring	Continuous large-scale scraping of billions of products and API access for premium data
PayPal Honey	Public (Free)	Automated coupon finder (primary), multi-retailer "Droplist" for tracking, and Amazon seller price comparisons	Browser extension tracking, affiliate-driven data model, and alleged "Secret Tab" redirection for attribution
37left.lk	Public (Free)	Multi-store price comparison specifically for Sri Lanka, 24/7 scanning, and WhatsApp/email alerts	Automated retailer scanning, lightning-fast internal search engine, and synonym matching
PriceGhost	Open-Source (Self-Hosted)	Universal price/stock tracking, interactive history charts, multi-user support, and back-in-stock alerts	"Price Voting" system using four parallel methods: JSON-LD, site-specific scrapers, generic CSS, and AI fallback
PriceBuddy	Open-Source (Self-Hosted)	Support for custom store addition (CSS/Regex), side-by-side multi-store comparison, and unit price calculation	Scrapers based on custom selectors/JSONPath, headless browser rendering (SeleniumBase), and AI self-healing
Omnia Retail	Enterprise	Multi-channel competitor monitoring, dynamic repricing, MAP enforcement, and Buy Box analytics	Real-time monitoring, direct API connections to comparison engines, and agentic AI for shift detection
Price2Spy	Enterprise	Specialized MAP monitoring with evidence collection, global marketplace tracking, and detailed history	4-tier matching (Manual, Automatch, Quick, ML Hybrid), hourly refresh rates, and manual QA validation
Competera	Enterprise	AI predictive pricing, high-accuracy exact and similar product matching, and demand elasticity modeling	Deep Learning for entity resolution, real-time client-specific crawling infrastructure, and AI pricing assistants

Table 2.8: Comparison of price-tracking and product-matching systems.

Chapter 3

Problem Specification and Requirements

3.1 Problem Specification

The problem specification for this project is to design and implement a framework that can efficiently crawl multiple e-commerce platforms, identify product pages, extract relevant product information, track historical prices, and compare products across different platforms. The framework should be scalable, efficient, and generalizable, allowing it to be used across different online platforms without the need for platform-specific rules or logic. The framework should also be able to identify the same product across different platforms, even if they are listed under different names or descriptions.

3.2 Aims and objectives

3.2.1 Aims

This project aims to address these limitations by introducing a framework that can be used to identify products and track their prices across multiple online platforms, while also providing historical price data to help consumers make informed decisions about their purchases. It will also include a product-matching algorithm that can identify the same product across different platforms, even if they are listed under different names or descriptions. The framework will be designed to be scalable and efficient, allowing for large-scale price tracking and cross-site product comparisons without incurring high costs or resource usage.

This framework will be generalizable, allowing it to be used across different online platforms, without the definition of rules or logic specific to any one platform.

While this framework is intended to be re-usable and adaptable to different product categories and platforms, this project will implement a proof-of-concept implementation, which will be a web-based application focused on Sri Lanka's e-commerce platforms, specifically focusing on consumer electronics.

3.2.2 Objectives

Primary objectives

- To design a distributed web crawler that can efficiently crawl multiple e-commerce platforms and collect page data.
- To evaluate different web-page classification techniques and use the most effective one to identify product pages from other types of pages.
- To evaluate different web-page information extraction techniques and use the most effective one to extract relevant product information from product pages.
- To implement a price-tracking framework to track historical prices for a given product at a specific retailer.
- To evaluate different product-matching techniques that can identify the same product across different platforms, and use the most effective one to implement a cross-site product comparison system.

Secondary objectives

- To implement a boilerplate identification and removal algorithm to minimize the amount of irrelevant data that needs to be stored and processed.
- To implement an observability framework to monitor the performance of the system and identify any issues that may arise.
- To implement a web-based application that allows users to search for products and view their historical prices across different platforms, as well as compare prices and identify the best deals available.

3.3 Requirements

3.3.1 Functional requirements

- Provided a set of seed URLs, the system must crawl each URL, render the page, and store its HTML content in a BLOB storage.
- For each page crawled, the system must identify hyperlinks to other pages and add them to the crawling queue.
- The system must be able to classify each crawled webpage as a product page or a non-product page.
- For each product page, the system must extract the product's title, current price, product image, and whether the product is in stock or out of stock.
- The system must be able to display the price-history of a product over time.
- Given a specific product from a specific e-commerce website, the system must be able to find the same product offered on other e-commerce websites.

3.3.2 Non-functional requirements

- The system must be able to track at least 100,000 products from at least 30 different e-commerce websites.
- The price of each product must be updated at least once every 72 hours.
- The distributed crawler must ensure that the crawling process does not impact the performance of the target e-commerce websites, and must respect the robots.txt file of each website.
- The system must be able to retrieve the price history for a product in under 500 milliseconds.
- The system must be able to find the same product offered on other e-commerce websites in under 2 seconds.
- The system must be implemented at a low cost of under USD 300 per month, including cloud hosting and storage costs.

Chapter 4

Methodology and Software Tools

4.1 Methodology

4.1.1 Approach

This project follows a prototype-driven methodology, which involves iterative development and testing of prototypes to refine the final product. The methodology consists of the following phases:

- **Web crawling and data collection:** The initial phase involves crawling the set of selected websites to store each page's HTML content for later use. This is done using a web crawler that systematically navigates through the websites and collects the necessary data. This phase always involves the use of a queue to manage the URLs to be crawled, ensuring that the crawler can efficiently handle large numbers of pages, while minimizing the number of duplicate pages crawled.
- **Data annotation:** A subset of the collected pages are then manually labelled to create a suitable machine learning model for future classification, information extraction, and product matching tasks. The pages were labelled based on the following characteristics:
  - **Page type:** Each page is classified as either a product page, a product list page, an error page, or other.
  - **Product information:** For product pages, relevant product information such as product name, price, and description is extracted and stored in a structured format.
- **Boilerplate removal:** Since the collected HTML pages contain a significant amount of boilerplate content (e.g., navigation menus, advertisements, and other non-essential elements), a boilerplate removal step is performed to extract the main content of each page. This is done in order to minimize the amount of noise in the data and improve the accuracy of subsequent machine learning models, as well as reduce the number of tokens to be processed, which can significantly improve the performance and resource usage of the models.
- **Page classification:** A machine learning model is trained to classify the pages into the two main categories: product pages and non-product pages. This model is trained using the manually labelled data from the previous phase, and is then used to automatically classify the remaining pages in the dataset.

- **Information extraction:** For the pages classified as product pages, a machine learning model is trained to extract relevant product information such as product name, price, and description. This model is also trained using the manually labelled data from the data annotation phase, and is then used to automatically extract product information from the remaining product pages in the dataset.
- **Product matching:** Finally, a product matching model is trained to identify and link identical products across different websites. This model is trained using the extracted product information from the previous phase, and is then used to automatically match products across different websites in the dataset.

Each phase of the methodology is iterative, with the results of each phase informing the next phase.

Finally, the results from all the phases are combined to create a **proof-of-concept (PoC) system**, which is a web-based application that allows users to search for products, analyze their price history, and compare prices across different websites.

Figure 4.1 illustrates the expected happy path that would be followed by a page in the project, going through each phase from crawling to its appearance in the PoC application.

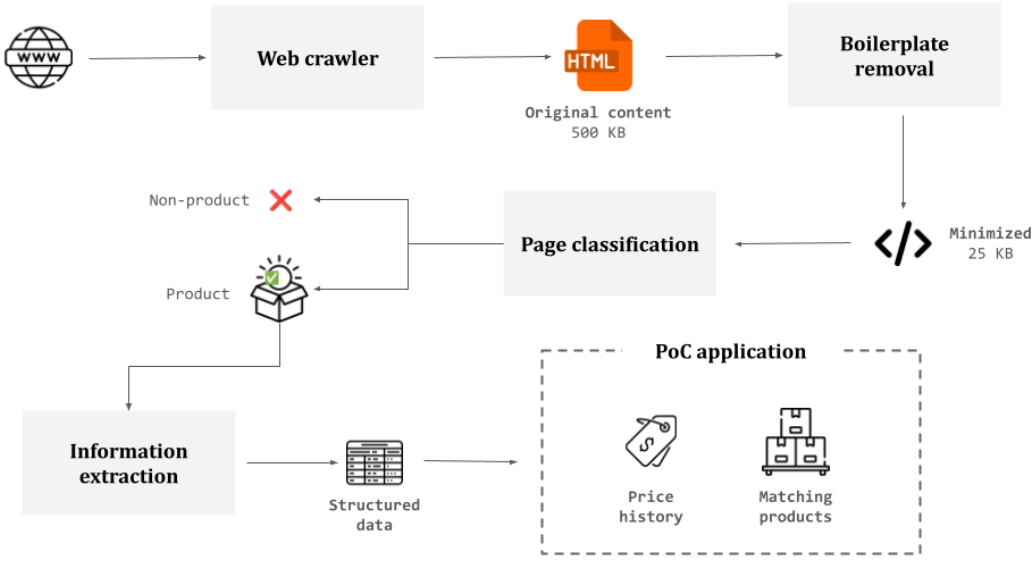

Figure 4.1: Lifecycle of a page

4.1.2 Evaluation

Each phase of the project is evaluated using different sets of qualitative metrics to assess the performance and effectiveness of the models and methods used. The evaluation metrics used for each phase are described in Table 4.1.

4.2 Software Tools

This section provides an overview of the software tools used in this research.

Phase	Evaluation metrics
Web crawler	• Number of pages crawled in 24 hours • Number of uncrawled pages in a 72-hour window • Average daily cost in USD
Data annotation	N/A
Boilerplate removal	• Average page size reduction (as a percentage of original content) • Average time and cost required to minimize 5000 pages
Page classification	• F1-score of the classification model on the validation dataset • Time and cost required to train and tune the model • Average time and cost required to classify 5000 pages
Information extraction	• Weighted accuracy of the model in successfully extracting the product title and price from the validation dataset • Time and cost required to train and tune the model • Average time and cost required to extract information from 5000 pages
Product matching	• Time and cost required to train and tune the model • F1-score of the model in product matching on the validation dataset
PoC application	• Average time (in ms) taken to retrieve the price history of a single product • Average time (in ms) taken to match similar products for a specific product

Table 4.1: Evaluation metrics used for each phase

4.2.1 Web crawling and data collection

- **Playwright**[1] was used to run headless browsers to render web pages and save the HTML content for further processing. Playwright provides several useful features such as pre-navigation and post-navigation hooks, which allow for the execution of custom code before and after page navigation. This is particularly useful for handling dynamic content and ensuring that all necessary data is captured. Another useful feature of Playwright is its ability to handle multiple browser contexts, which allows for the simulation of different user sessions and the collection of data from multiple sources simultaneously. With this we can load multiple pages in parallel, which significantly speeds up the data collection process. Playwright also provides a powerful API for interacting with web pages, which allows for the automation of tasks such as hiding certain elements, clicking buttons, and filling out forms. Finally, Playwright also allows to define a target concurrency level as well as automatic throttling and retries, which helps to avoid overloading the target servers and ensures that the data collection process is efficient and reliable.

- **Amazon Simple Queue Service (SQS)**[2] was used to manage the queue of URLs to be crawled. SQS is a fully managed message queuing service that enables the decoupling and scaling of microservices, distributed systems, and serverless applications. SQS provides several useful features such as message visibility timeout, which allows for the temporary hiding of messages from other consumers while they are being processed, and message retention, which allows for the storage of messages for a specified period of time. Another useful feature was the ability to set up dead-letter queues, which allows for the handling of messages that cannot be processed successfully.

[1] `https://playwright.dev/`
[2] `https://aws.amazon.com/sqs/`

4.2.2 Boilerplate removal

- **BeautifulSoup**[3] is a Python library for parsing HTML and XML documents. This library was used as it offers a simple and intuitive API for navigating and manipulating the parse tree, which makes it easy to extract and modify the content of web pages. BeautifulSoup also provides several useful features for handling malformed or poorly structured HTML, which is common in web pages, and allows for the extraction of specific elements or attributes from the HTML content.

4.2.3 Proof-of-concept application

- **Next.js**[4] was used to build the front-end of the proof-of-concept application. Next.js is a popular React framework that provides several useful features for building web applications, such as server-side rendering, static site generation, and automatic code splitting. These features help to improve the performance and scalability of the application, as well as providing a better developer experience.

4.2.4 Data storage and management

- **Amazon Simple Storage Service (S3)**[5] was used to store the crawled HTML content, minimized html content, model training data and the trained model artifacts. S3 is a fully managed object storage service that provides several useful features such as high durability, scalability, and availability. S3 also provides several storage classes, which allow for the optimization of storage costs based on the access patterns and retention requirements of the data. Another useful feature of S3 is its integration with other AWS services, which allows for the seamless transfer of data between different components of the application.

- **Aurora PostgreSQL on Amazon Relational Database Service (RDS)**[6] was used to store the metadata of the crawled web pages, such as the URL, title, and description. PostgreSQL also offers support for advanced data types including vectors, which is useful for storing and querying the embeddings generated by the product matching model.

- **Google BigQuery**[7] was used as the data source for the observability dashboard, which allows for the monitoring of the different components of the application and provides insights into the performance and health of the system. BigQuery is a fully managed, serverless data warehouse that provides several useful features such as high scalability, fast query performance, and integration with other Google Cloud services. BigQuery was selected for this task, due to its deep integration with Google Data Studio, which allows for the easy creation of interactive dashboards and reports, performance with analytical workloads, and well as due to its generous free tier, which allows for the storage and querying of large datasets at almost no cost.

[3] `https://www.crummy.com/software/BeautifulSoup/`
[4] `https://nextjs.org/`
[5] `https://aws.amazon.com/s3/`
[6] `https://aws.amazon.com/rds/aurora/postgresql/`
[7] `https://cloud.google.com/bigquery`

4.2.5 Containerization and orchestration

- **Docker**[8] was used to containerize the different components of the application, which allows for the isolation of dependencies and the easy deployment of the application across different environments.
- **Amazon Elastic Container Registry (ECR)**[9] was used to store the Docker images of the different components of the application, which allows for the easy distribution and deployment of the application.
- **Amazon Elastic Container Service (ECS)**[10] was used to orchestrate the deployment and scaling of the different components of the application, which allows for the efficient management of resources and the automatic scaling of the application based on demand. This was primarily used for the crawler deployment, which allows for the automatic scaling of the number of crawler instances based on the number of URLs in the SQS queue. Using spot instances for the crawler deployment also helps to reduce the cost of running the crawler, as spot instances are generally cheaper than on-demand instances.

4.2.6 Model training and inference

- **Amazon SageMaker**[11] was used to train and deploy the product classification, information extraction, and product matching models. A unique feature offered by Sagemaker is its feature to run training jobs with hyperparameter tuning, which allows for the automatic optimization of the model hyperparameters based on a specified objective metric.
- **Google Colab**[12] was used for prototyping and experimentation with different model architectures and training strategies. Google Colab provides a very user-friendly interface for running Jupyter notebooks in the cloud with access to free GPU and TPU resources, which allows for the efficient training of small machine learning models.
- **PyTorch**[13] was used to implement the machine learning models for product classification, information extraction, and product matching. PyTorch is a popular open-source machine learning framework that provides several useful features for building and training deep learning models, such as dynamic computation graphs, automatic differentiation, and a rich ecosystem of pre-trained models and libraries. PyTorch also provides several useful tools for model debugging and visualization, which helps to improve the interpretability and explainability of the models.
- **Hugging Face Transformers**[14] was used to implement the transformer-based models for product classification, information extraction, and product matching. Hugging Face Transformers is a popular open-source library that provides several pre-trained transformer models and tools for fine-tuning and deploying these models for various natural language processing tasks. The library also provides several useful features for model training and evaluation, such as data preprocessing, tokenization, and model evaluation metrics, which helps to improve the efficiency and effectiveness of the model training and evaluation process.

[8] https://www.docker.com/
[9] https://aws.amazon.com/ecr/
[10] https://aws.amazon.com/ecs/
[11] https://aws.amazon.com/sagemaker/
[12] https://colab.research.google.com/
[13] https://pytorch.org/
[14] https://huggingface.co/transformers/

4.2.7 Programming languages

- **TypeScript**[15] was used for several components of this project, such as the crawler and the web-interfaces. TypeScript is a common language used for web development (especially front-end), and it provides several useful features such as static typing, interfaces, and classes, which help to improve the maintainability and scalability of the codebase. TypeScript, as a superset of JavaScript, also allows for the use of modern JavaScript features and libraries, which helps to improve the performance and functionality of the application.
- **Python**[16] was used for several components of this project, especially those related to machine learning and model training and inference. Python is a popular programming language for data science and machine learning, and it provides several useful libraries and frameworks for these tasks, such as NumPy, Pandas, Scikit-learn, TensorFlow, and PyTorch. These libraries and frameworks provide a wide range of functionality for data manipulation, analysis, and modeling, which helps to improve the efficiency and effectiveness of the machine learning components of the project.
- **Golang**[17] was used for several components of this project, especially those related to the backend APIs and data processing. Golang is a statically typed, compiled programming language that provides several useful features for building high-performance and scalable applications, such as concurrency, garbage collection, and a simple syntax. Golang also provides a rich standard library and a growing ecosystem of third-party libraries and frameworks, which helps to improve the efficiency and effectiveness of the backend components of the project. Golang compiled programs generally have a smaller memory footprint and faster execution time compared to interpreted languages like Python, which helps to improve the performance and scalability of the backend components of the project.

4.2.8 Infrastructure as Code (IaC)

- **Terraform**[18] was used to manage the infrastructure of the application as code. Terraform is an open-source tool that allows for the definition and provisioning of infrastructure resources in a declarative manner, which helps to improve the reproducibility and maintainability of the infrastructure. Terraform also provides several useful features such as state management, which allows for the tracking of changes to the infrastructure over time, and modules, which allow for the reuse of infrastructure code across different projects and environments.

4.2.9 Source code management

- **GitHub**[19] was used to manage the source code for all the components of the application. GitHub provides several useful features such as version control, which allows for the tracking of changes to the source code over time, and collaboration, which allows for the sharing of code and the coordination of development efforts among multiple contributors. GitHub also provides several integrations with other tools and services, such as continuous integration and deployment (CI/CD) pipelines, which

[15] `https://www.typescriptlang.org/`
[16] `https://www.python.org/`
[17] `https://golang.org/`
[18] `https://www.terraform.io/`
[19] `https://github.com`

allows for the automatic testing and deployment of the application whenever changes are pushed to the repository.

4.2.10 Continuous Integration and Continuous Deployment (CI/CD)

- **GitHub Actions**[20] was used to automate the CI/CD pipelines for the application. GitHub Actions allows for the definition of workflows that can be triggered by various events, such as pushes to the repository or the creation of pull requests, which helps to improve the efficiency and reliability of the development process. This was used to orchestrate pipelines which automatically build, test, and deploy the different components of the application whenever changes are pushed to the repository.

4.2.11 Other tools

- **AWS Amplify**[21] was used to manage the deployment and hosting of the web applications related to this project. Amplify allows you to connect your application to GitHub repositories and automatically deploy the application whenever changes are pushed to the repository.
- **AWS Lambda**[22] was used to run serverless functions for various short-lived tasks such as handling API requests and managing the crawler queue.
- **AWS Step Functions**[23] was used to orchestrate the different stages of the model inference pipeline, which allows for the efficient management of resources and automatic retries in case of failures.
- **Google Data Studio**[24] was used to create the observability dashboard for the application. Data Studio provides several useful features for creating interactive dashboards and reports, such as drag-and-drop interface, data connectors, and visualization options.
- **Google Datastream**[25] was used to replicate the data from Aurora PostgreSQL to Google BigQuery, viawhich allows for the efficient querying and analysis of the data in the observability dashboard.

[20] `https://github.com/features/actions`
[21] `https://aws.amazon.com/amplify/`
[22] `https://aws.amazon.com/lambda/`
[23] `https://aws.amazon.com/step-functions/`
[24] `https://datastudio.google.com/`
[25] `https://cloud.google.com/datastream`

Chapter 5

System design

The proposed system consists of eight different modules, each of which is responsible for a specific task. The modules are interconnected and work together to achieve the overall functionality of the system. The modules are as follows:

1. The **Distributed web crawler** module is responsible for crawling the selected websites and collecting the HTML content of each page. It uses a distributed architecture to efficiently handle large numbers of pages and minimize duplicate crawls. Each page is repeatedly crawled on a frequent basis to ensure that the data is up-to-date.
2. The **HTML minimizer** module is responsible for removing boilerplate content from the crawled HTML pages, retaining only the relevant content for further processing.
3. The **Data labeller** is a simple web application that enabled easy annotation of the crawled pages. It allows users to label pages based on their type (product page, product list page, error page, or other) and relevant product information (product name, price, image, and stock status).
4. The **Page classifier** module is responsible for classifying the crawled pages into product pages and non-product pages.
5. The **Information extractor** module is responsible for extracting relevant product information from the product pages, such as product name, price, image, and stock status.
6. The **Product matching** module is responsible for identifying and linking identical products across different websites.
7. The **Proof-of-concept (PoC) system** is a web-based application that allows users to search for products, analyze their price history, and compare prices across different websites.
8. The **Observability dashboard** is dashboard which allows for the monitoring of the above modules' performance and health, providing insights into the system's operation and enabling proactive maintenance.

The following sections provide a detailed description of each module and its functionality.

5.1 Database design

The primary database used for this project is a PostgreSQL database, hosted on Amazon RDS. PostgreSQL is a powerful, open-source relational database management system that provides robust features for data storage, retrieval, and management. It is chosen for its reliability, scalability, and support for extensive data types and extensions, making it suitable for handling the requirements of this project.

5.1.1 Database schema

The database schema was designed to cover the requirements of each of the phases of the data pipeline outlined in the system design. The schema consists of several tables, each serving a specific purpose in the data pipeline.

The main tables in the database schema are as follows, and the schema diagram is shown in figure 5.1:

- **crawler_pages:** This table stores the metadata associated with each crawled (or to be crawled) page. This table it primarily used to keep track of the crawling process, and is used to populate the crawler queue. In the later stages of the pipeline, it used to identify recently crawled pages to be passed to the page classification and information extraction modules.
- **minimized_pages**: This table keeps track of the metadata associated with the page minimization phase. Similar to the `crawler_pages` table, it is used to keep track of the minimization process and identify recently minimized pages to be passed to the page classification and information extraction modules.
- **page_classifications**: This table stores the current results of the page classification phase, including the classification of each page as a product page or non-product page.
- **product_information**: This table stores the extracted product information from the product pages, including the product name, price, image, and stock status.
- **price_history**: This table stores the historical price data for each product, allowing for analysis of price trends and identification of genuine sales and discounts. This table is linked to the `product_information` table via Change Data Capture (CDC) to ensure that any changes to the price in `product_information` table are reflected in the `price_history`.
- **product_embeddings**: This table stores the embeddings of the product pages, which are used for product matching across different websites. The embeddings are generated using a pre-trained model and are stored in this table for efficient retrieval and comparison.

Aside from the main tables, there are two auxiliary tables which are only used for the model training phases, and are not connected to the components in the system or the other tables in the database schema. These tables are as follows and their schema diagram is shown in figure 5.2:

- **page_labels**: This table stored the annotated labels for the crawled pages, which are used for training the page classification model as well as the information extraction model.

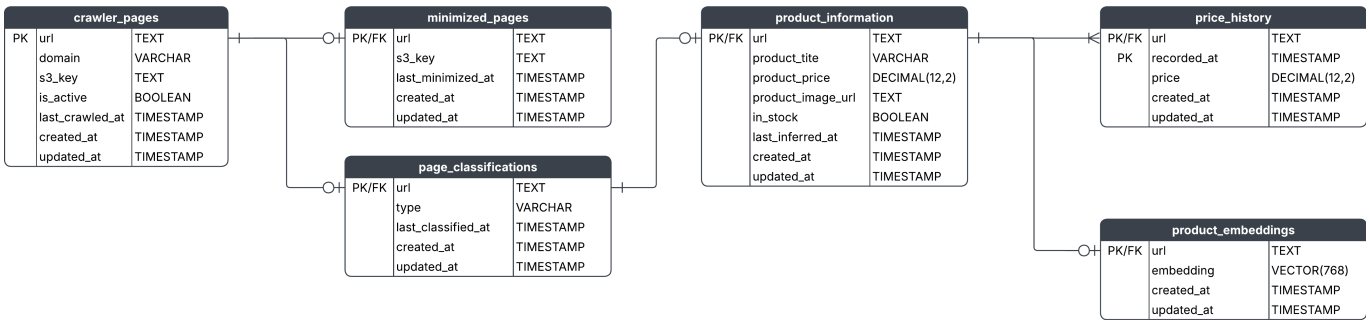

Figure 5.1: Database schema diagram

- **matching_products**: This table consists of pairs of matching and non-matching products, which are used for training the product matching embedding model.

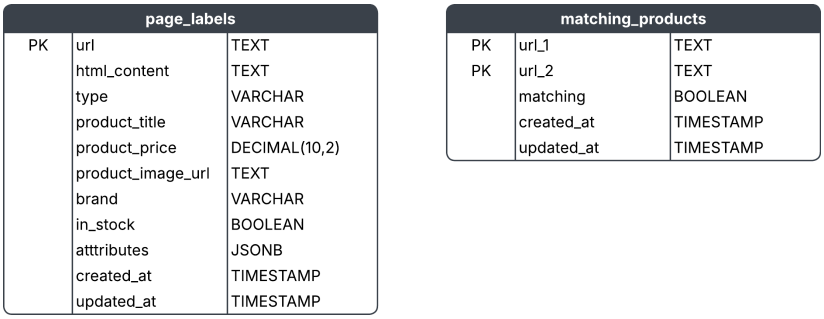

Figure 5.2: Database auxiliary schema diagram

5.2 Distributed crawler

The distributed crawler forms the starting point of the system design. It is responsible for fetching and processing web pages from the set of e-commerce websites and storing them in order to be processed in the later stages. The crawler is designed to operate across multiple nodes, allowing it to scale horizontally and handle large volumes of data efficiently. It also has built-in mechanisms for retries and exponential backoff to handle transient errors and avoid overwhelming the target websites with requests.

The crawler is implemented with the use of three subcomponents.

1. **Crawler queue:** This is an Amazon SQS queue that holds the URLs to be crawled. It allows for decoupling the crawling process from the processing of the crawled data, enabling better scalability and fault tolerance.

2. **Queue manager:** This is a lambda function which runs every 15 minutes, and is responsible for populating the crawler queue with new URLs to be crawled.

3. **Crawler workers:** These are Amazon ECS tasks which loads URLs from the crawler queue, fetches the corresponding web pages, and saves their content.

5.2.1 Crawler queue

The crawler queue is a fairly simple component. It is implemented using Amazon SQS, which provides a reliable, scalable, and fully managed message queuing service. The queue holds the URLs to be crawled, allowing the crawler workers to fetch and process them asynchronously. This decoupling of the crawling process from the processing of the crawled data enables better scalability and fault tolerance.

It consists of two queues: a main queue and a dead-letter queue. The main queue holds the URLs to be crawled, while the dead-letter queue holds the URLs that could not be processed successfully after a certain number of retries. This allows for better error handling and monitoring of the crawling process, as failed URLs can be analyzed and reprocessed later.

The main queue is configured with a visibility timeout of 1 hour, which means that once a URL is fetched by a crawler worker, it will not be visible to other workers for 1 hour. This allows the worker to have enough time to process the URL and avoid duplicate processing. Once a URL is successfully processed, it is deleted from the queue, ensuring that it is not processed again. Moreover, the main queue is configured with a maximum message retention period of 1 day, which means that if a URL is not processed within 1 day, it will be automatically deleted from the queue. This prevents the queue from growing indefinitely and ensures that only relevant URLs are kept for processing.

5.2.2 Queue manager

The queue manager is a lambda function that runs every 15 minutes and is responsible for populating the crawler queue with new URLs to be crawled. It first checks whether the crawler queue is empty, and if it is not, it does nothing and exits. If the queue is empty, it checks the database for any URLs which match at least one of the following criteria:

- The URL has not been crawled even once.
- The URL is classified as a product page, but has not been crawled in the last 60 hours.
- The URL is classified as a non-product page, but has not been crawled in the last 7 days.

The queue manager fetches the URLs that match these criteria and adds them to the crawler queue for processing by the crawler workers. This ensures that the crawler is continuously fetching and processing new data, while also re-crawling existing data to keep it up-to-date. The figure 5.3 shows the logic of the queue manager in a flowchart format.

The behaviour of the crawler ensures that each product page is crawled at least once every 3 days, while all other pages are crawled at roughly once every 7 days.

5.2.3 Crawler workers

The crawler workers are Amazon ECS tasks that are responsible for fetching and processing the web pages from the crawler queue. Each worker loads a URL from the queue, fetches the corresponding web page, and saves its content to the database. The workers are designed to operate in parallel, allowing for efficient

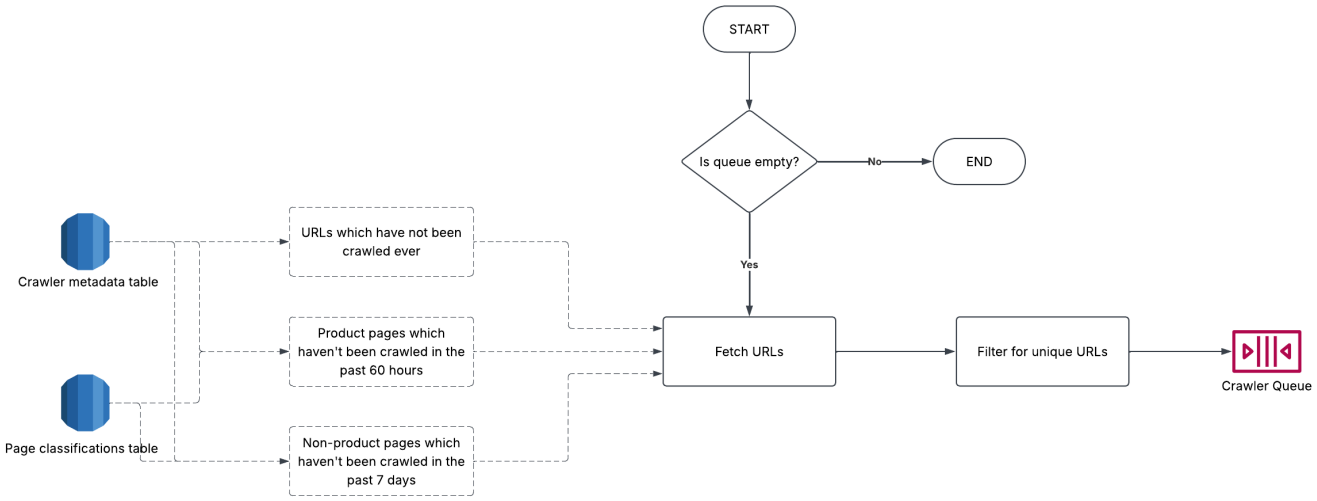

Figure 5.3: Crawler queue manager flow diagram

processing of large volumes of data. They also have built-in mechanisms for retries and exponential backoff to handle transient errors and avoid overwhelming the target websites with requests.

For most of the lifecycle of a processed page, the JavaScript library `Playwright` is used to interact with the page and retrieve its content. Playwright is a powerful library that allows for automated browser interactions, enabling the crawler to handle dynamic content and JavaScript-heavy pages effectively. It also supports pre-navigation and post-navigation hooks, which allow for custom logic to be executed before and after the page is loaded. The figure 5.4 presents the crawler worker logic as a flowchart.

For each URL fetched by a worker, the following actions are performed:

- Checks if the URL belongs to an approved domain. If it does not, the URL is discarded and not processed further.
- The URL is compared against known blacklisted patterns, and if it matches any of them, it is discarded and not processed further.
- Checks the URL against the site's `robots.txt` file to ensure that the crawler is allowed to access the page. This is done to avoid crawling pages that are known to be irrelevant or problematic, such as login pages, checkout pages, malicious links, or pages that are known to cause issues with the crawler.
- Retrieves the initial content of the page using an HTTP GET request.
- Checks if the initial HTTP response is a redirect, and if so, follows the redirect to the final destination URL.
- Block the loading of images, external stylesheets, advertisements, and other non-essential resources via pre-navigation hooks to reduce bandwidth usage, minimize the load on the target servers, and speed up the crawling process.
- Waits up to 15 seconds for the page to load completely, including any dynamic content that may be generated by JavaScript.
- Downloads the final HTML content of the page and

  Saves it to an Amazon S3 bucket for further processing.

Updates the crawler metadata table in the database with the status of the crawl and the last crawled timestamp.

- Checks whether the page contains any JSON-LD structured data, and if so,

  Check whether the page is a product page based on the JSON-LD data, and updates the page classification table in the database accordingly.

  Extracts the relevant product information from the JSON-LD data and saves it to the product information tables in the database.

- Detects all the outgoing hyperlinks on the page and adds them to crawler metadata table in the database for future crawling.

- Removes URL from the crawler queue to ensure that it is not processed again.

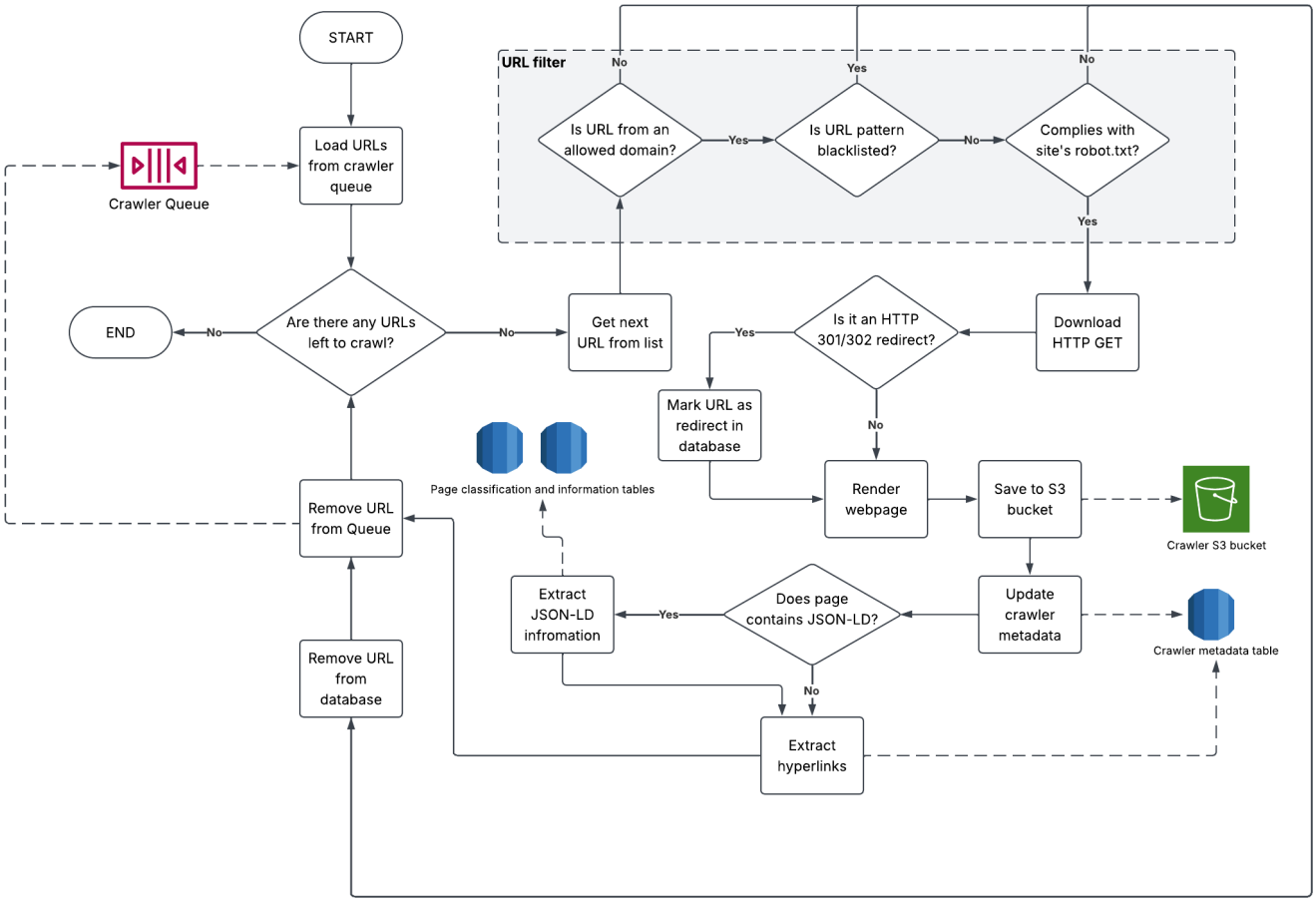

Figure 5.4: Crawler worker flowchart

The crawler workers are run as tasks on AWS Fargate Spot, which allows for cost-effective and scalable execution of containerized applications. However since they are run on spot instances, they can be interrupted at any time by AWS. To handle this, the crawler workers are designed to be stateless and idempotent, meaning that they can be safely restarted without losing any data or causing inconsistencies. If a worker is interrupted, it will simply stop processing the current URL and return it to the crawler queue for processing by another worker. As a result, each worker instance only crawls a small number of URLs, but there can be many worker instances running in parallel, allowing for efficient processing of large volumes of data. This ensures that the crawling process can continue uninterrupted, even in the face of spot instance interruptions. If it is required to increase the rate of crawling, the number of worker instances can be increased, allowing for more URLs to be processed in parallel.

5.2.4 Crawler system architecture

The figure 5.5 presents the overall architecture of the distributed crawler system. It shows the interactions between the different components, including the crawler queue, queue manager, and crawler workers, as well as the database and S3 bucket used for storing the crawled data.

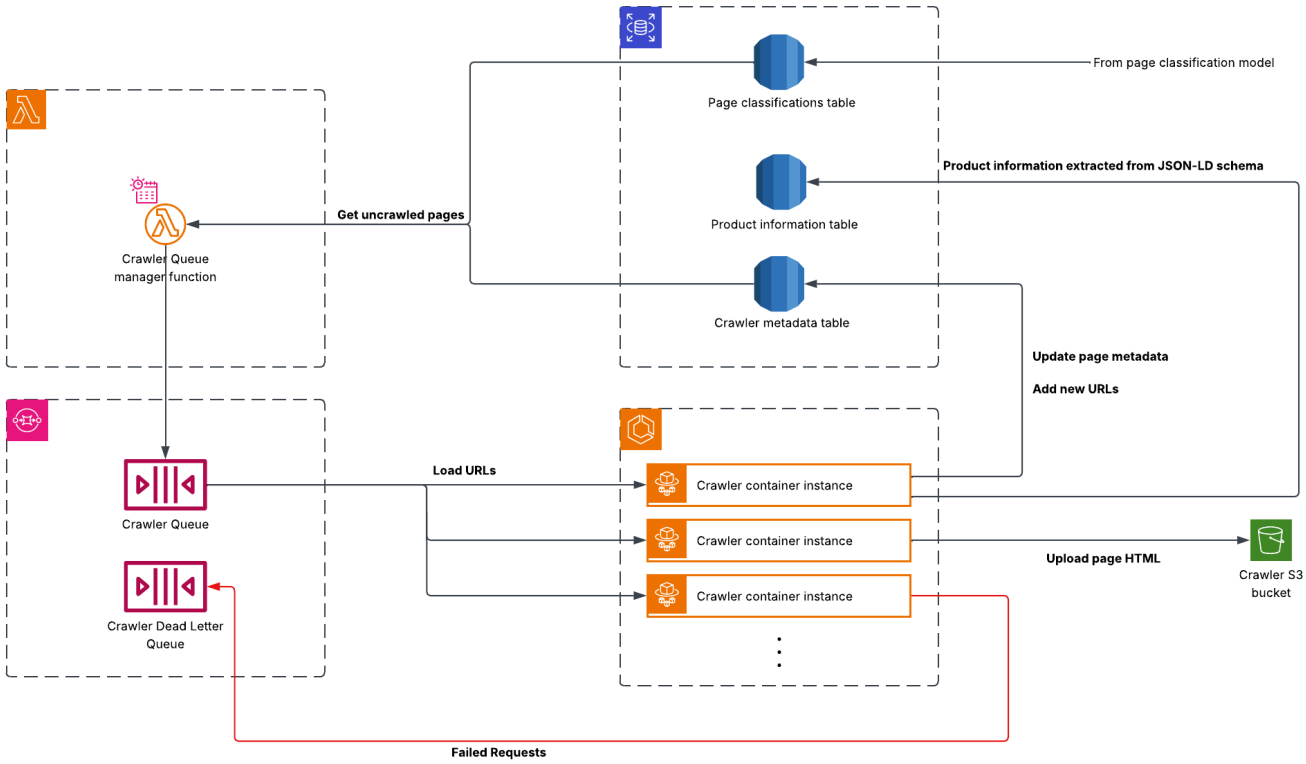

Figure 5.5: Distributed crawler system architecture

5.3 HTML minimizer

The HTML minimizer module is responsible for removing boilerplate content from the crawled HTML pages, retaining only the relevant content for further processing. This module is crucial for improving the efficiency and accuracy of the subsequent page classification and information extraction tasks. The minimizer works by analyzing the structure of the HTML page and identifying the main content areas, while discarding headers, footers, advertisements, and other non-essential elements. This process reduces the amount of data that needs to be processed and helps focus the extraction algorithms on the most relevant parts of the page. The benefits of minimizing a page before processing include:

- **Improved accuracy:** By removing irrelevant content, the minimizer helps the subsequent modules focus on the most relevant parts of the page, leading to more accurate classification and extraction results.
- **Reduced processing time:** Minimizing the HTML pages reduces the amount of data that needs to be processed, which can lead to faster processing times and improved system performance.
- **Reduced cost:** Minimizing the HTML pages reduces the amount of data that needs to be processed, which can lead to lower operational costs.

- **Lower storage usage:** By reducing the size of the HTML pages, the minimizer can help lower memory and storage requirements, making the system more efficient and cost-effective.

The HTML minimizer module consists of three main subcomponents:

- **Boilerplate removal library**: This is a library written in Python, which implements a variation of the site style trees algorithm for boilerplate removal. This package is published on PyPI[1] and can be installed using pip.
- **Anchor tree generation**: This is a process which generates anchor trees for each domain in the dataset.
- **HTML minimizer workers**: These are AWS ECS tasks which run on a schedule. On each run, they fetch all pages which have been crawled but not yet minimized, and process them using the boilerplate removal library. The minimized pages are then stored in an S3 bucket for further processing by the page classifier, information extractor, and product matching modules.

5.3.1 Boilerplate removal library

This library implements a variation of the site style trees algorithm for boilerplate removal. Given the HTML content from a web page, the library first runs a pre-processing step to clean the HTML and remove any unnecessary elements, such as scripts and styles. Next, it constructs a site style tree for each page, which represents the hierarchical structure of the page's content. Figure 5.6 illustrates how a site style tree is constructed from the HTML content of a web page.

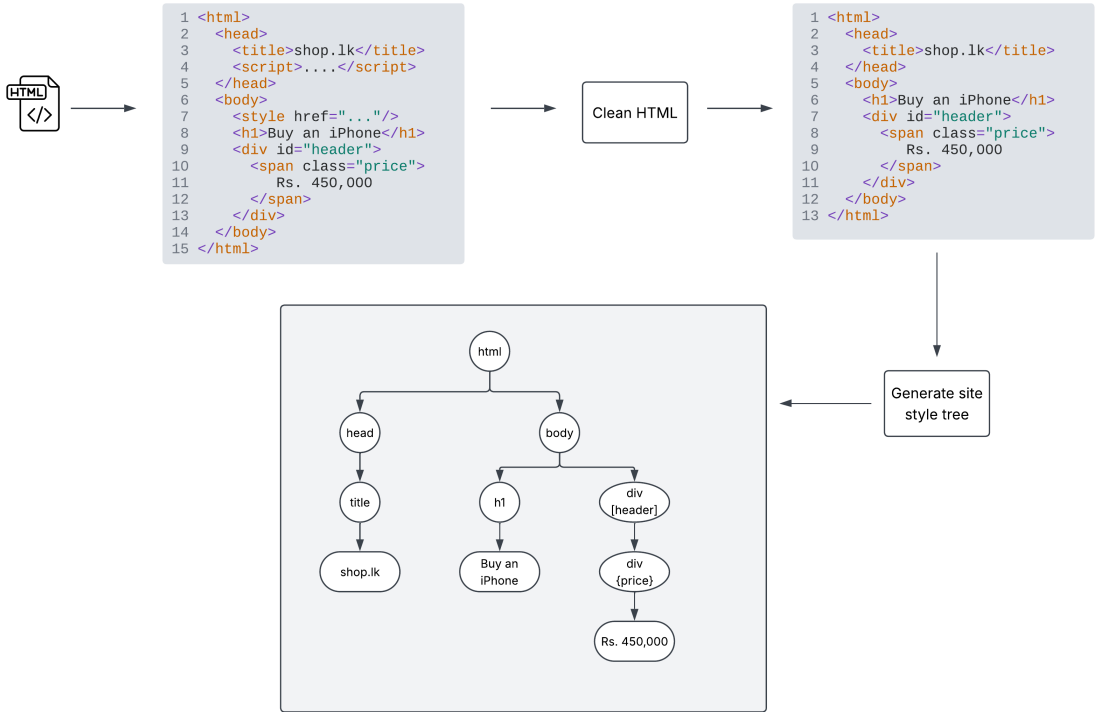

Figure 5.6: Simple example of site style tree construction from HTML content.

[1] https://pypi.org/project/boilerplate-remover/

Then, these individual site style trees are merged to create a single site style tree that captures the common structure of the pages.

Each node in the tree contains the following attributes:

- **Tag name:** The HTML tag name of the node (e.g., div, p, h1).
- **ID:** The unique identifier of the node, if any (e.g., id attribute).
- **Text content:** The text content of the node, if any.
- **Children:** A list of child nodes, representing the nested structure of the HTML elements.
- **Count:** The number of times this node appears in the set of HTML pages.

If a node has a high count, it indicates that the node is common across multiple pages and is likely to be boilerplate content.

Finally, all nodes with a count below a certain predefined threshold are removed from the tree, as they are considered to not be boilerplate content. The remaining tree structure then becomes the anchor tree, which is a representation of the common structure of the pages. Figure 5.7 illustrates how an anchor tree is constructed from a collection of site style trees.

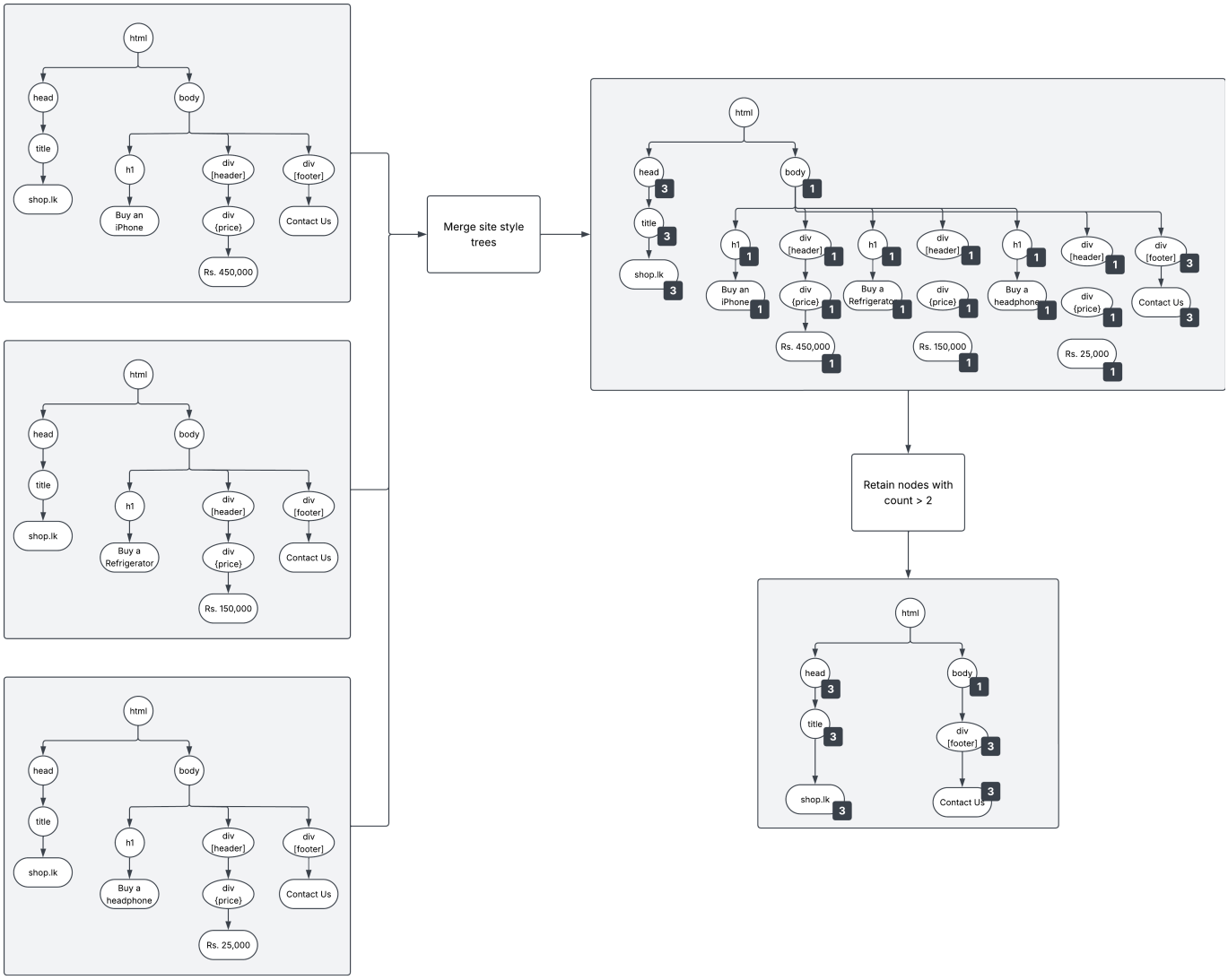

Figure 5.7: Simple example of anchor tree construction from a collection of site style trees.

When a new page is processed, its site style tree is compared to the anchor tree. Nodes that match the anchor tree are considered to be boilerplate content and are removed from the page. The remaining nodes are considered to be the main content of the page and are retained. The processed tree is passed to a post-processing step, which reconstructs the HTML page with only the relevant content, and the minimized HTML is returned. Figure 5.8 illustrates how a minimized page is generated from a site style tree and an anchor tree.

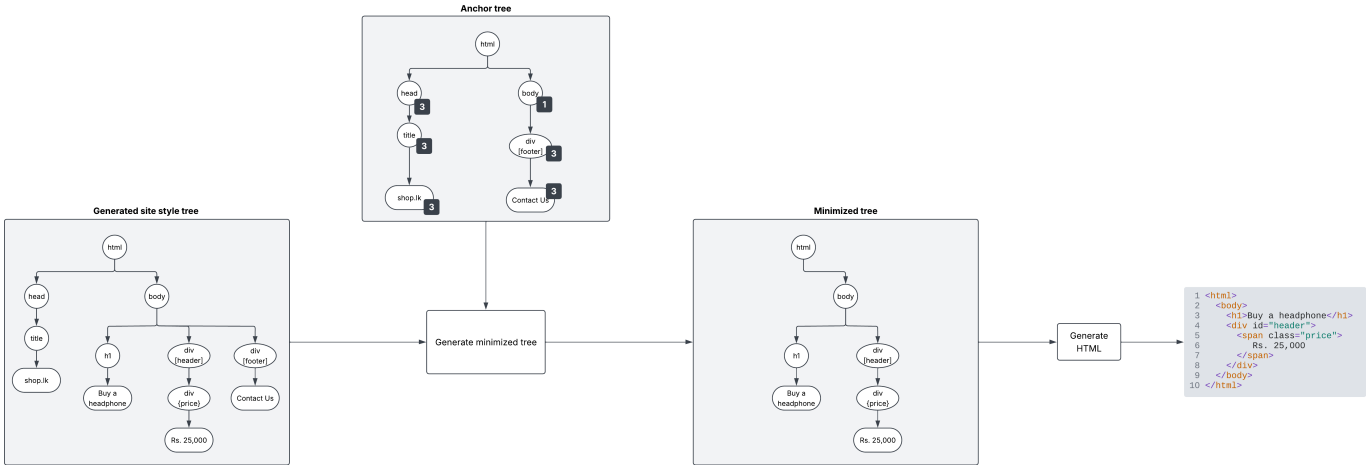

Figure 5.8: Simple example of minimized page generation from a site style tree and an anchor tree.

5.3.2 Anchor tree generation

As described above, anchor trees are generated for each domain by taking a random sample of 100 pages from the domain and constructing site style trees for each page. These anchor trees are then stored in an S3. Figure 5.9 illustrates how anchor trees are generated and stored for each domain in the dataset.

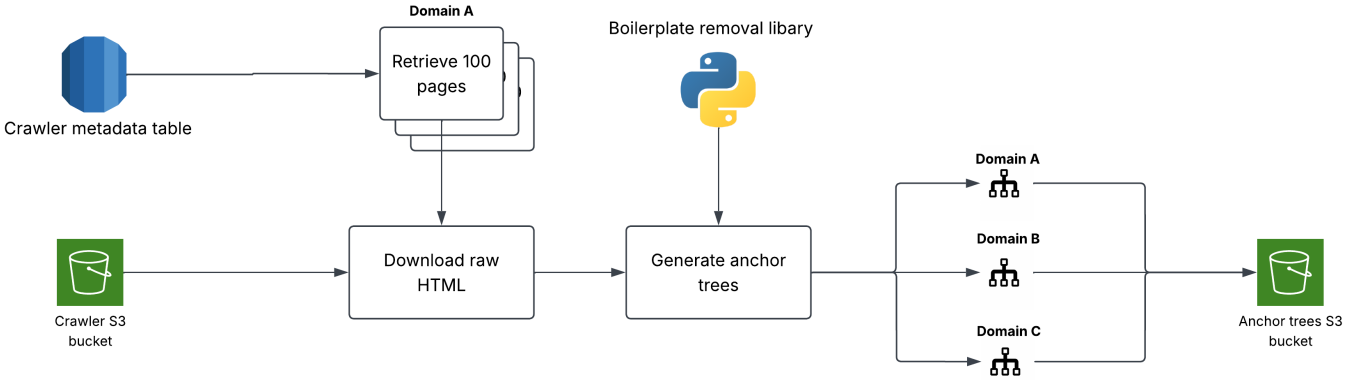

Figure 5.9: How anchor trees are computed for each domain.

5.3.3 HTML minimizer workers

The HTML minimizer workers are AWS ECS tasks which implement the above boilerplate removal library and well as the pre-generated anchor trees to generate minimized versions of recently crawled pages. The workers run on a schedule, fetching all pages which have been crawled but not yet minimized in batches of 20,000 pages per run.

On each run, the workers perform the following steps:

1. Fetch the list of pages crawled but not yet minimized from the database.
2. For each page, fetch the HTML content from the S3 bucket.

3. For each page, fetch the corresponding anchor tree for the domain from the S3 bucket.
4. For each page, run the boilerplate removal library to generate a minimized version of the HTML page.
5. Store the minimized HTML page in an S3 bucket.
6. Update the database to mark the page as minimized.

Figure 5.10 illustrates how an HTML minimizer worker processes crawled pages to generate minimized versions of the HTML pages.

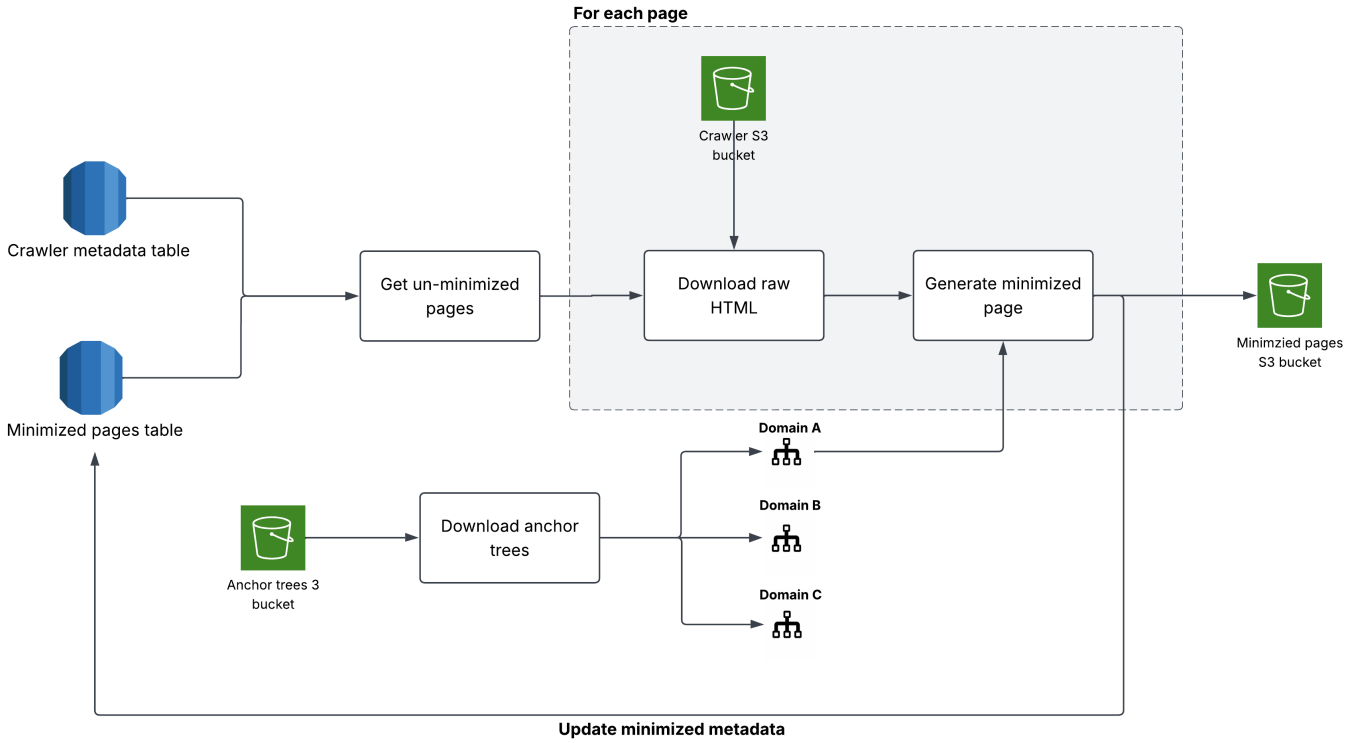

Figure 5.10: Execution flow of the HTML minimizer workers.

5.3.4 HTML minimizer architecture

Figure 5.11 illustrates the architecture of the HTML minimizer module, which consists of the boilerplate removal library, anchor tree generation, and HTML minimizer workers, and how they interact with the database and S3 buckets to process crawled pages and generate minimized versions of the HTML pages.

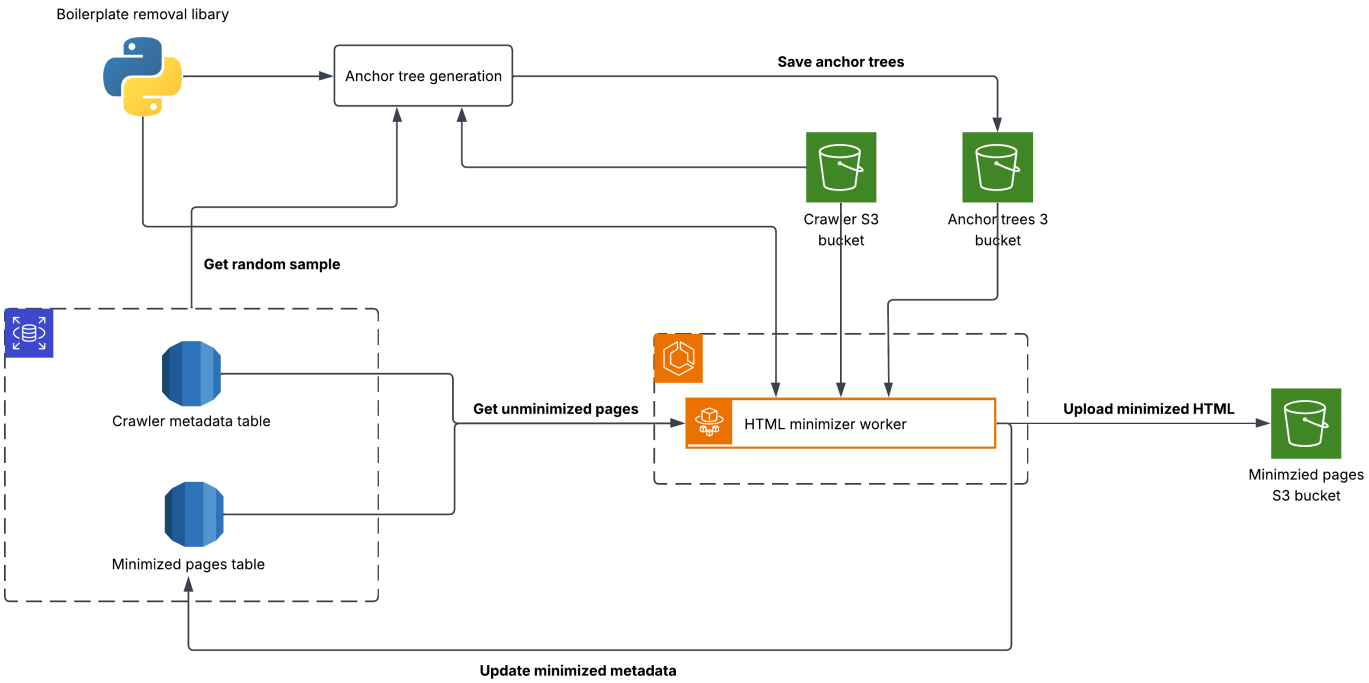

Figure 5.11: Architecture of the HTML minimizer module.

5.4 Data annotation application

This is a simple web-application which provides a user interface for users to label the crawled pages. On load, it will present a randomly selected page from the crawled set of pages, and the user can first classify it as a product page, product list page, error page, or other. If the page is classified as a product page, the user can then annotate the relevant product information, including the product title, image, price, brand, and stock status.

Figure 5.12 shows a screenshot of the data annotation application, which is designed to be simple and intuitive to use, allowing users to quickly and accurately label the crawled pages. The application shows two side-by-side views of the page, with the first being the original page and the second being a minimized view of the page, which is generated by the HTML minimizer module. On the right side of the page, there are a set of input fields for the user to enter the relevant product information, and a set of buttons to classify the page and submit the annotations. Once submitted, the annotated data will be stored in the `page_labels` table in the database, along with the HTML content of the corresponding page. The annotated data will be used to train the page classifier and information extractor modules, which will enable the system to automatically classify and extract relevant product information from the crawled pages.

This application is developed with Next.js, a React framework for building server-side rendered web applications. Next.js was selected for this task as it allows for the quick development of simple, uncomplex web applications, and provides a good developer experience. The web application is hosted on AWS Amplify, which provides a simple and scalable hosting solution for web applications and is connected to the GitHub repository for continuous deployment.

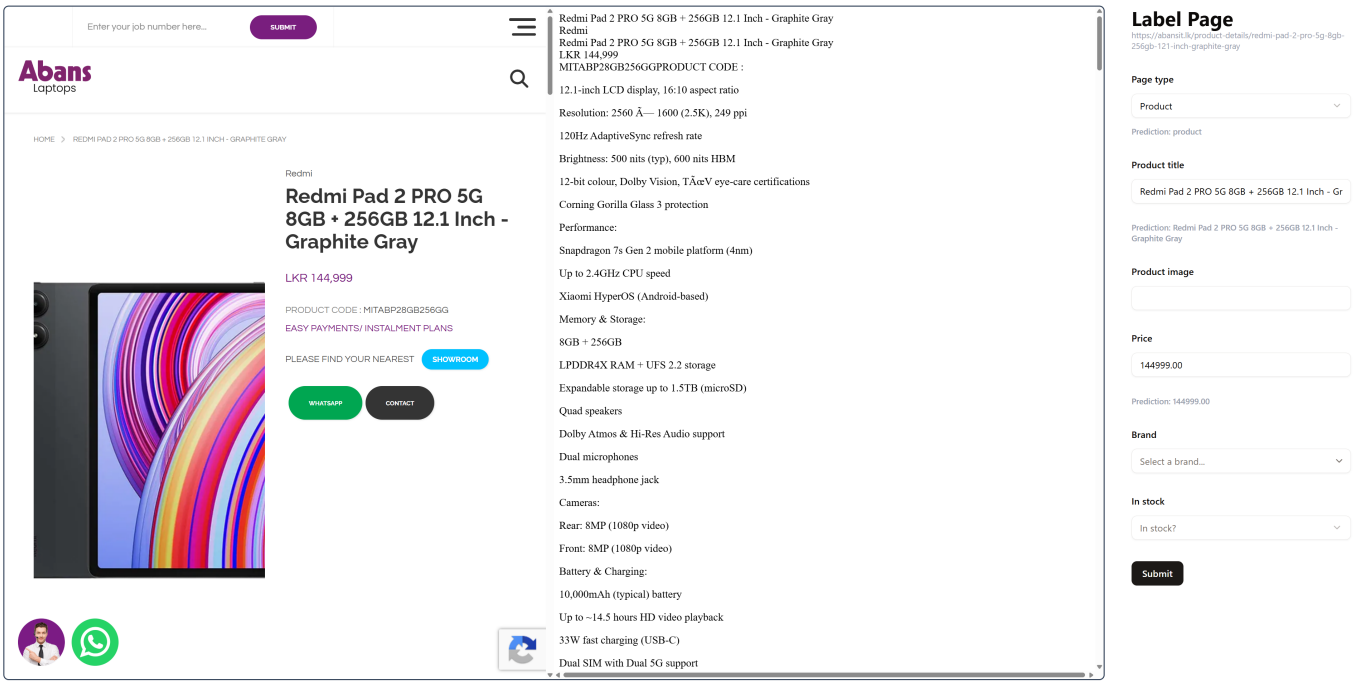

Figure 5.12: Screenshot of the data annotation application.

5.5 Page classifier

The page classifier module is responsible for classifying the crawled pages into product pages and non-product pages - which is essentially a binary classification problem. This module is crucial for filtering out irrelevant pages and ensuring that only product pages are processed by the information extractor and product matching modules. The page classifier is able to accurately identify product pages by analyzing the content and structure of the minimized HTML pages.

This takes place in three phases.

1. **Training phase:** In this phase, the page classifier is trained using the labelled dataset of HTML pages. The labelled dataset is created using the data annotation application. The training phase involves model selection, and model training and fine-tuning.

2. **Model evaluation and selection:** In this phase, the trained models are evaluated using a separate validation dataset, and the best performing model is selected for deployment. Other factors such as training/inference speed and cost are also considered when selecting the best model.

3. **Inference phase:** In this phase, the trained page classifier is used to classify new crawled pages into product pages and non-product pages.

5.5.1 Training phase

Since the data is unstructured and the HTML pages can vary significantly in terms of content and structure, a deep learning approach is used for the page classification task. For this purpose, three different model architectures were trained and evaluated.

- **TF-IDF with traditional machine learning classifiers:** This approach involves extracting TF-IDF features from the HTML content and using traditional machine learning classifiers, such as logistic regression, support vector machines (SVM), and random forests, to classify the pages.
- **BERT embeddings with a neural network classifier:** This approach involves using a pre-trained BERT model to generate embeddings for the textual content of the page, which are then fed into a neural network classifier for classification.
- **MarkupLM for document classification:** This approach involves using a pre-trained MarkupLM model, which is specifically designed for processing HTML documents, to classify the pages.

In each of the above approaches, the model's hyperparameters were tuned using a grid search approach available in AWS SageMaker, which is illustrated in Figure 5.13.

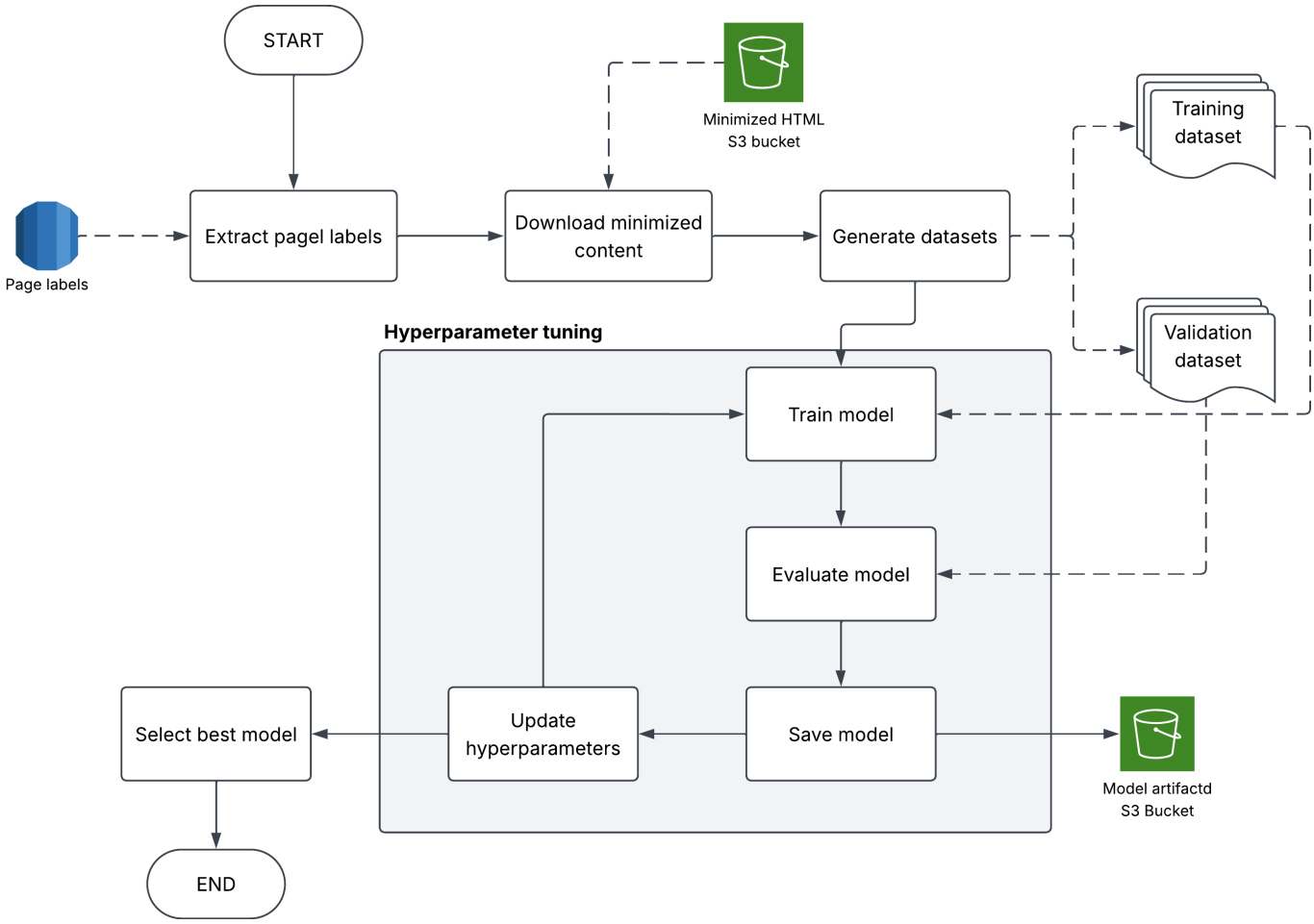

Figure 5.13: Overview of the page classification model training phase.

5.5.2 Model evaluation and selection

The results of the best performing model for each approach are summarized in Table 5.1. While the F1-score is the primary metric for evaluating the model's performance, other factors such as training and inference speed, as well as cost, were also considered when selecting the best model for deployment.

Based on the results in Table 5.1, the BERT embeddings with a neural network classifier approach was selected for deployment in the inference phase, as it provides a good balance between accuracy, speed, and

Technique	F1 score	Pros	Cons
TF-IDF with Random Forest Regressor	0.930	• Very fast training and inference (˜3 mins for 20,000 classifications) • Extremely cheap inference (˜$0.03 for 20,000 classifications)	• Low performance
BERT Embeddings with Neural Network	0.963	• Decent training and inference speed (˜6 mins for 20,000 classifications) • Somewhat cheap inference (˜$0.22 for 20,000 classifications)	• Requires a GPU-accelerated machine
MarkupLM classification	0.989	• Extremely accurate	• Requires a GPU-accelerated machine • Slow training and inference (˜13 mins for 20,000 classifications) • Expensive (˜$0.40 for 20,000 classifications)

Table 5.1: Comparison of page classification approaches

cost.

5.5.3 Inference phase

The inference phase involves using the best performing model from the training phase to classify new crawled pages as product pages or non-product pages. This is implemented as a scheduled batch job that runs once a day. In each run, the following steps are performed:

1. Fetch the list of recently crawled pages that have not yet been classified from the database.
2. For each page, fetch the minimized HTML content from the S3 bucket.
3. For each page, generate BERT embeddings from the minimized HTML content.
4. For each page, use the trained neural network classifier to classify the page as a product page or a non-product page.
5. Store the classification results in the database, along with the relevant metadata, such as the page URL and the timestamp of the classification.

This pipeline is implemented using AWS Step Functions, which orchestrates the different steps of the pipeline and ensures that they are executed in the correct order. It also provides error handling and retry mechanisms to ensure that the pipeline can recover from failures (especially due to unavailability of compute resources in AWS) and continue processing the remaining stages. Figure 5.14 illustrates the overall architecture of the page classification inference phase.

5.5.4 Page classifier architecture

The page classifier architecture is designed to be scalable and efficient, allowing for the processing of large volumes of crawled pages in a timely manner. Figure 5.15 illustrates the overall architecture of the page

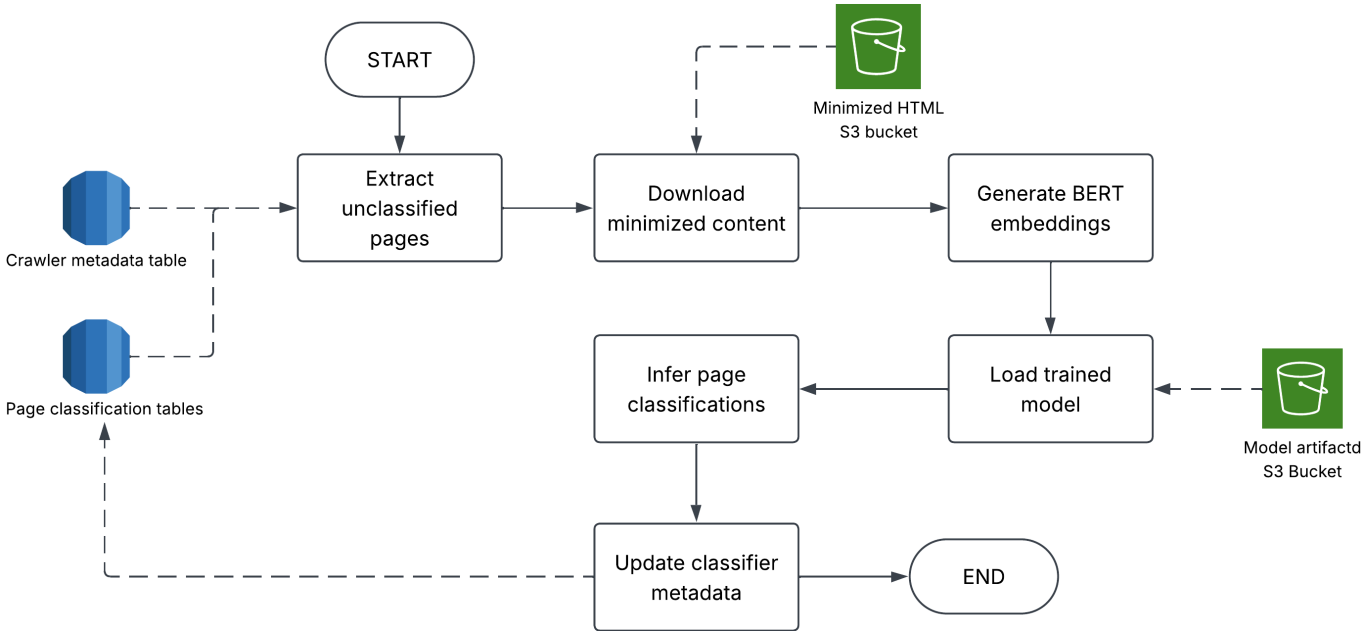

Figure 5.14: Overview of the page classification inference phase.

classifier module, which includes the different components involved in the training and inference phases, as well as the interactions with the database and S3 buckets.

5.6 Information extraction

The information extraction module is responsible for extracting relevant product information from the product pages, such as the name of the product, its price, image, and availability. Similar to the page classifier module, the information extraction module is also trained using a labelled dataset of HTML pages, which is created using the data annotation application. The information extraction module is able to accurately extract product information by analyzing the content and structure of the minimized HTML pages.

The information extraction process takes place in three phases.

1. **Training phase:** In this phase, different information extraction models are trained/fine-tuned using the labelled dataset of HTML pages. The training phase involves model selection, and model training, fine-tuning, and hyperparameter tuning.

2. **Model evaluation and selection:** In this phase, the trained models are evaluated using a separate validation dataset, and the best performing model is selected for deployment. Other factors such as training/inference speed and cost are also considered when selecting the best model.

3. **Inference phase:** In this phase, the trained information extraction model is used to extract relevant product information from newly crawled product pages.

4. **Price history tracking**: In this phase, whenever a product's information is extracted, its price is recorded with the timestamp so that its historical price can be tracked.

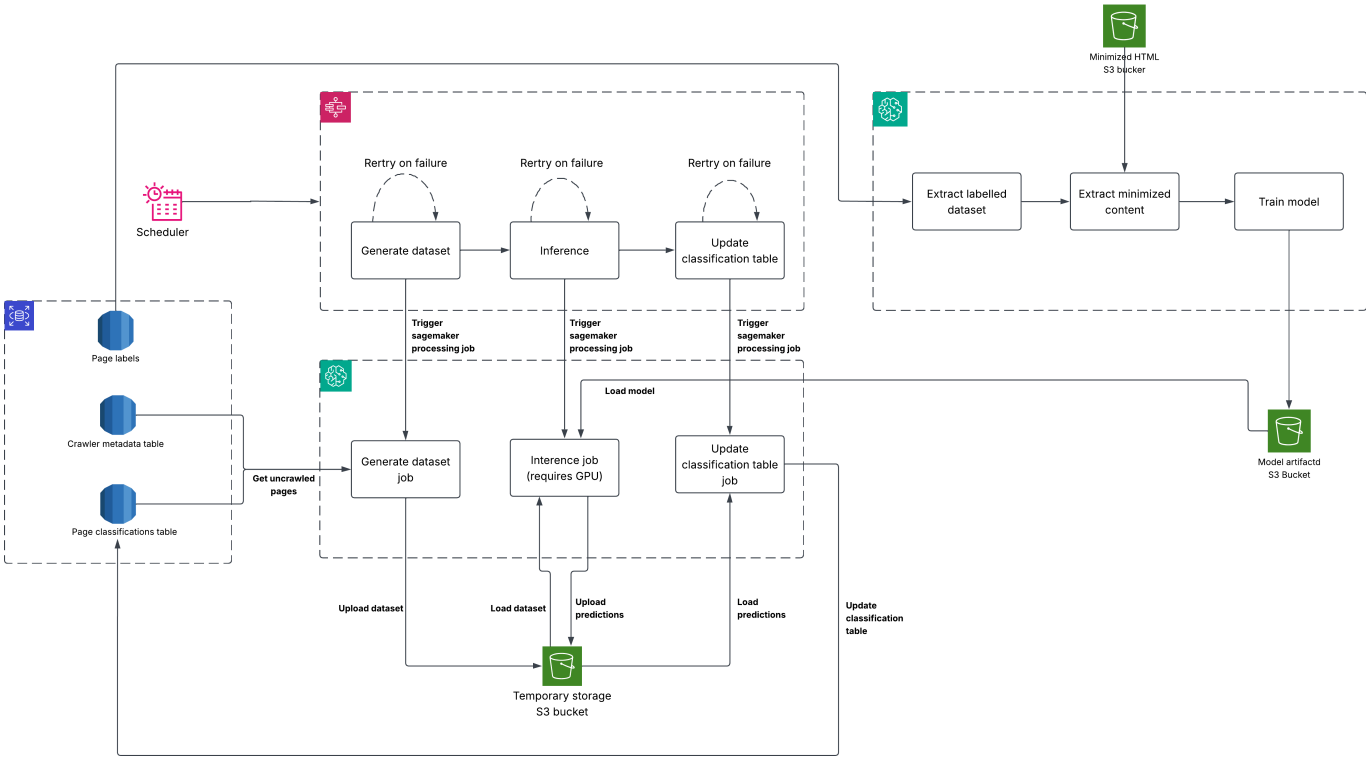

Figure 5.15: Overview of the page classifier architecture.

5.6.1 Training phase

Since the dataset is once again unstructured, consisting of HTML pages with varying content and structure, a deep learning approach is used for the information extraction task as well. For the information extraction task, three different techniques were trained and evaluated.

- **MarkupLM for token classification:** This approach involves using a pre-trained MarkupLM model, which is specifically designed for processing HTML documents, to classify the tokens in the HTML content into different product information categories (e.g., name, price, image, availability).
- **Multi-modal approach with LayoutLMv3:** This approach involves using a multi-modal model that combines textual and visual information from the HTML pages to extract product information.
- **LLM based approach:** This approach involves using a large language model (LLM) to extract product information from the HTML pages by leveraging its natural language understanding capabilities.

Using MarkupLM for token classification

MarkupLM is a pre-trained model that is specifically designed for processing HTML/XML documents, and it has been shown to perform well on various document understanding tasks. With this approach, the task essentially becomes a token classification problem, where each token in the HTML content is classified into one of the product information categories.

- `TITLE-TOKEN`: This category represents the tokens that belong to the product title.
- `PRICE-TOKEN`: This category represents the tokens that belong to the product price.

- `IMAGE-TOKEN`: This category represents the tokens that belong to the product image URL.
- `OUT-OF-STOCK-TOKEN`: This category represents the tokens that indicate that the product is out of stock.
- `NULL-TOKEN`: This category represents the tokens that do not belong to any of the above categories.

The OUT-OF-STOCK-TOKEN was selected as most sites do not explicitly mention the availability of a product, but rather indicate that a product is out of stock when it is unavailable for purchase. This token is identified by the presence of certain keywords in the HTML content, such as "out of stock", "unavailable", "sold out", etc.

The implementation of this approach first involved the conversion of the labelled dataset into a format suitable for token classification, where each token in the HTML content is assigned a label corresponding to its category. This was done viaà custom pre-processing script that tokenizes the HTML content of a minimized page and assigns labels to each token based on the annotated product information in the labelled dataset. For example, for the product title, there would be two types of tokens named `B-TITLE` and `I-TITLE`, where `B-TITLE` represents the first token of the product title and `I-TITLE` represents the subsequent tokens of the product title. Figure 5.16 illustrates how the pre-processing script would assign labels to the tokens in the HTML content of a minimized page based on the annotated product information in the labelled dataset.

Figure 5.16: Example of how a product page is tokenized with the `B-` and `I-` classes

With this tokenized dataset, the MarkupLM model was fine-tuned for the token classification task using the labelled dataset of HTML pages. Similar to the page classification model, the hyperparameters of the MarkupLM model were tuned using a grid search approach available in AWS SageMaker. Figure 5.17 illustrates how the MarkupLM model was fine-tuned for the token classification task.

During the training phase, it was observed that the MarkupLM model did not perform well at classifying the tokens correctly. This waa primarily due to the high cardinality of the token classes, with two classes for each product information category (e.g., `B-TITLE` and `I-TITLE` for the product title), which made it difficult for the model to learn the correct token classifications. The solution was to drop the `I-` classes and

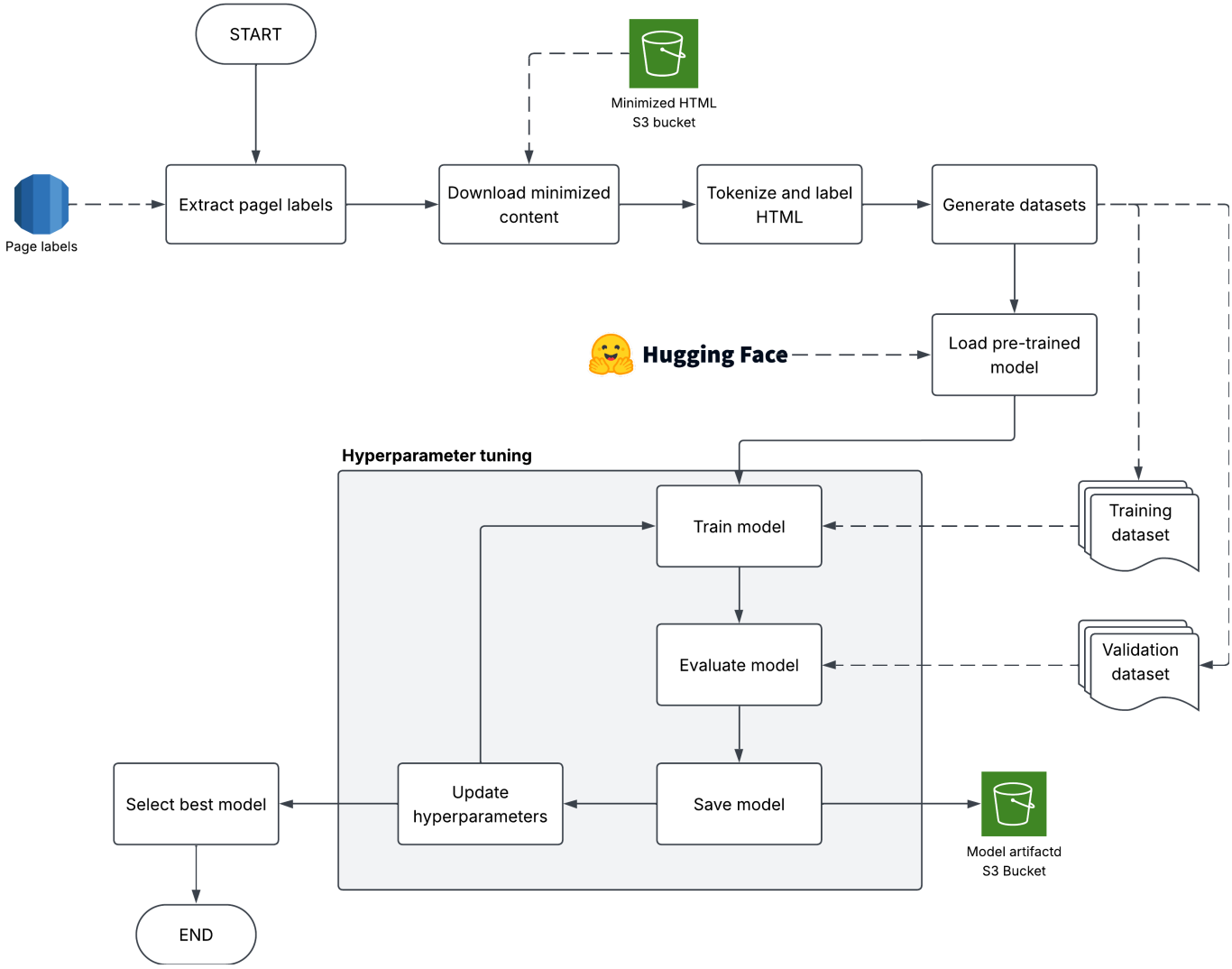

Figure 5.17: Overview of the information extraction model training phase.

keep only the `B-` classes for each product information category, which reduced the number of token classes and improved the model's performance. Figure 5.18 illustrates how the pre-processing script would assign labels to the tokens in the HTML content of a minimized page based on the annotated product information in the labelled dataset, with only the `B-` classes.

With this tokenized dataset, the performance of the model improved significantly, and the model was able to accurately classify the tokens in the HTML content into the correct product information categories.

Multi-modal approach with LayoutLMv3

This approach involved using a multi-modal model that combines textual and visual information from the HTML pages to extract product information. The LayoutLMv3 model was used for this purpose, which is a pre-trained model that is specifically designed for document understanding tasks that involve both textual and visual information.

Similar to the MarkupLM approach, there involved a pre-processing step which combined the HTML content of a page with the labelled product information in the labelled dataset to create a multi-modal dataset suitable for training the LayoutLMv3 model.

Figure 5.18: Example of how a product page is tokenized with only the `B-` classes

1. The original HTML content (not minimized) of a product page was converted into an image using a headless browser, which rendered the HTML content and captured a screenshot of the page.
2. The labelled product information in the labelled dataset was used to create bounding boxes around the relevant product information in the image of the product page. The bounding boxes were created using the coordinates of the HTML elements corresponding to the labelled product information.
3. The bounding boxes were then labelled with the corresponding product information category (e.g., name, price, image, availability) to create a multi-modal dataset suitable for training the LayoutLMv3 model.

Figure 5.19 illustrates how the pre-processing script would create bounding boxes around the relevant product information in the image of a product page based on the annotated product information in the labelled dataset.

After this pre-processing stage, the LayoutLMv3 model was fine-tuned for the information extraction task using the labelled dataset of HTML pages, similar to the MarkupLM approach. The hyperparameters of the LayoutLMv3 model were also tuned using a grid search approach available in AWS SageMaker.

LLM based approach

The LLM based approach was comparatively simpler than the other two approaches, as it did not require any pre-processing of the HTML content or the training/fine-tuning of a model. Instead, the LLM was used to extract product information from the HTML content of a product page by providing it with a prompt that describes the task and the expected output format. Two LLM models were evaluated for this approach, Google's Gemini 2.5 Flash Lite and Google's Gemini 2.5 Flash. The LLM was able to accurately extract product information from the HTML content by leveraging its natural language understanding capabilities.

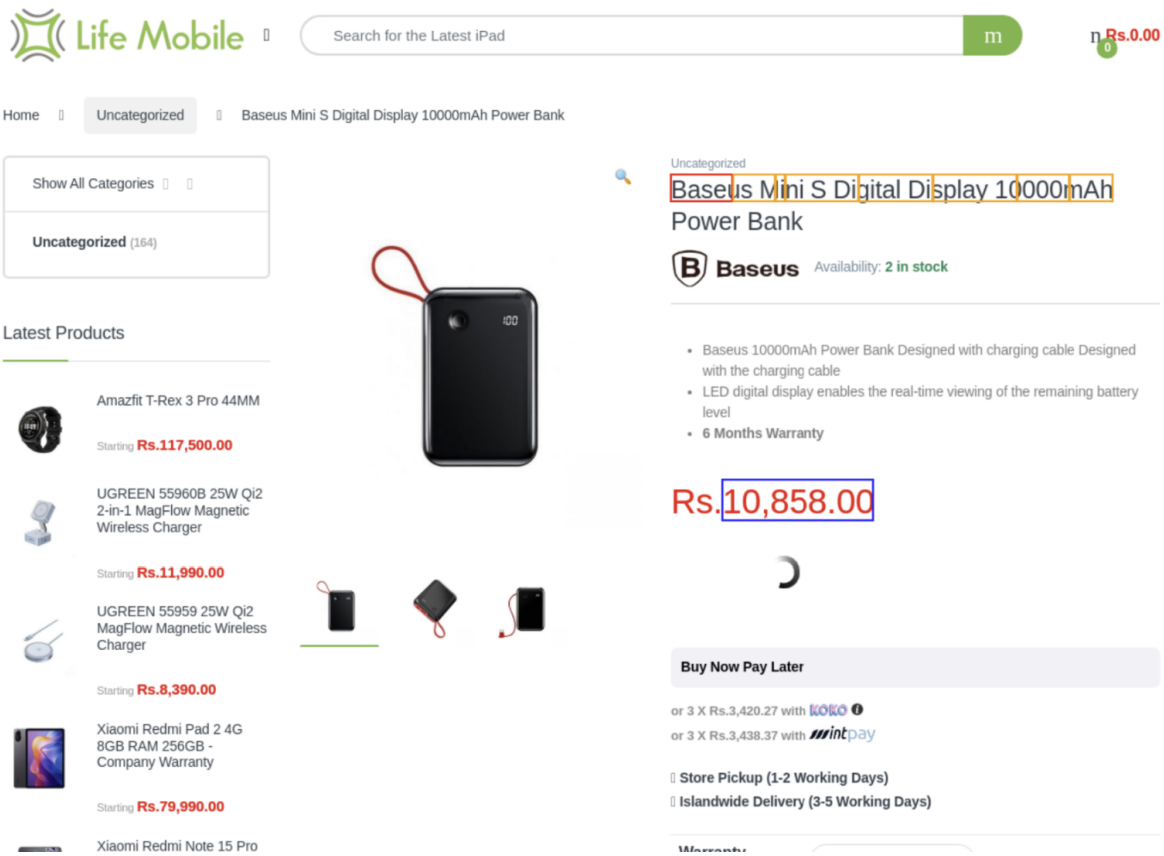

Figure 5.19: Example of bounding boxes created around the relevant product information in the image of a product page

5.6.2 Model evaluation and selection

The three approaches were evaluated using a separate validation dataset, and the best performing model was selected for deployment. For this phase, only the extraction of the product title and price was considered, as the extraction of the product image and availability was not a priority for the proof-of-concept system.

The comparative results are summarized in Table 5.2.

Technique	**Title Accuracy**	**Price Accuracy**	**Pre-processing time & cost (for 20,000 pages)**	**Extraction time & cost (for 20,000 pages)**
MarkupLM	0.951	0.920	22 mins $0.14	7 mins $0.18
Multi-modal	0.866*	0.785*	121 mins $0.88	13 mins $0.34
LLM (Gemini 2.5 Flash Lite)	0.935	0.970	10 mins $0.04	883 mins** $85.85**
LLM (Gemini 2.5 Flash)	0.975	0.960	10 mins $0.04	1,576 mins** $281.68**

Table 5.2: Comparison of information extraction approaches

All models were trained and then tested against data from websites they had not been exposed to.

* The performance of the multi-modal model could be improved with further training and fine-tuning, but not enough to justify the additional cost.

** Extraction time and cost for the LLM approaches are estimated based on a sample of 200 pages.

** Extraction time for the LLM approaches could be improved through parallelization.

Although the LLM based approach achieved the highest accuracy for both title and price extraction, it was not selected for deployment due to its high inference time and cost. The MarkupLM approach was selected for deployment as it achieved a good balance between accuracy, inference time, and cost, making it suitable for the information extraction task in the proposed system.

5.6.3 Inference phase

In the inference phase, the trained MarkupLM model is used to extract relevant product information from newly crawled product pages. The inference process involves the following steps:

1. Extract the newly crawled product pages which have not yet been processed by the information extraction model.
2. For each page, download the minimized HTML content from the S3 bucket.
3. Pre-process the HTML content to tokenize it and assign labels to each token based on the product information categories.
4. Feed the tokenized HTML content into the fine-tuned MarkupLM model to classify the tokens into the product information categories.
5. Post-process the output of the MarkupLM model to extract the relevant product information (e.g., name, price, image, availability) from the classified tokens.
6. Store the extracted product information in the database, along with the relevant metadata, such as the page URL and the timestamp of the extraction.

The post-processing step intends to solve several problems that arise from the token classification output of the MarkupLM model.

- **Only the beginning of the product information is classified:** Since only the `B-` classes were used for token classification, the MarkupLM model only classifies the beginning of the product information (e.g., the first token of the product title or price).
- **Cleaning the extracted product information:** The output of the MarkupLM model may contain extraneous tokens that are not part of the actual product information (e.g., HTML tags, special characters, etc.). These extraneous tokens are removed during the post-processing step to obtain clean product information. The post-processing step involves extracting the complete product information by concatenating the subsequent tokens until the end of the parent HTML element is reached.
- **Multiple tokens identified for the same class:** In some cases, the MarkupLM model may classify multiple tokens as belonging to the same product information category (e.g., multiple tokens classified

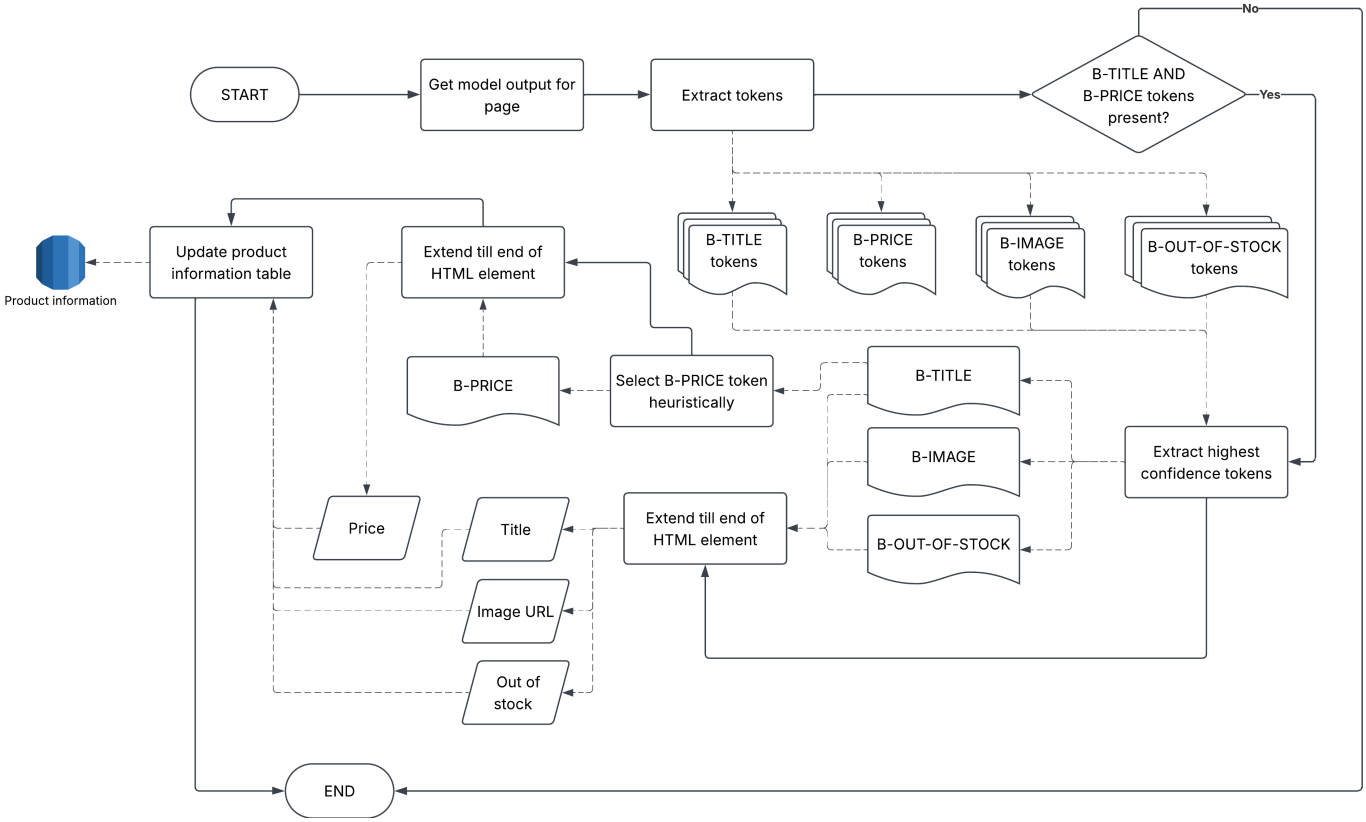

Figure 5.20: Post-processing flow for information extraction

as `B-TITLE`). This is solved by simply selecting the token with the highest confidence score for each product information category.

- **Identification of pre-sales price and installment prices:** In some cases, the MarkupLM model may classify tokens as belonging to the `B-PRICE` category, but the identified price may not be the actual selling price of the product. For example, some websites may display a pre-sales price or an installment price instead of the actual selling price. In such cases, the post-processing step selects the best token through a heuristic approach, which involves selecting the correct token based on its proximity to the selected `B-TITLE` token and the value of the expanded price (e.g., a price which is 1/3rd of one of the other visible prices are considered to be installment prices).
- **No tokens identified for a class:** In some cases, the MarkupLM model may not classify any tokens as belonging to a particular product information category (e.g., no tokens classified as `B-PRICE`). In cases such as this, it is assumed that the page classification model has misclassified the page as a product page, and the page is ignored.

Figure 5.20 illustrates the post-processing flow and how the correct values for the various product information categories are computed.

5.6.4 Price history tracking

Price history tracking is implemented by using Change Data Capture (CDC) on the product information table. Whenever a product is updated, its new price is recorded with the last crawled timestamp in the price history table. This is implemented by using PostgreSQL triggers as shown in Listing 5.1. With this

approach, a historical record of price snapshots of all products are tracked.

```
CREATE OR REPLACE FUNCTION record_price_change()
    RETURNS TRIGGER AS
$$
BEGIN
    INSERT INTO product_price_history (url, price, recorded_at)
    VALUES (NEW.url, NEW.product_price, NOW());

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_record_price_changes ON page_inferred_labels;

CREATE TRIGGER trg_record_price_changes
    AFTER INSERT OR UPDATE
    ON page_inferred_labels
    FOR EACH ROW
EXECUTE FUNCTION record_price_change();
```

Listing 5.1: Trigger used to record product price changes

5.6.5 Information extraction module architecture

Figure 5.21 illustrates the architecture of the information extraction module, which includes the training phase, inference phase, and price history tracking and how they interact with each other.

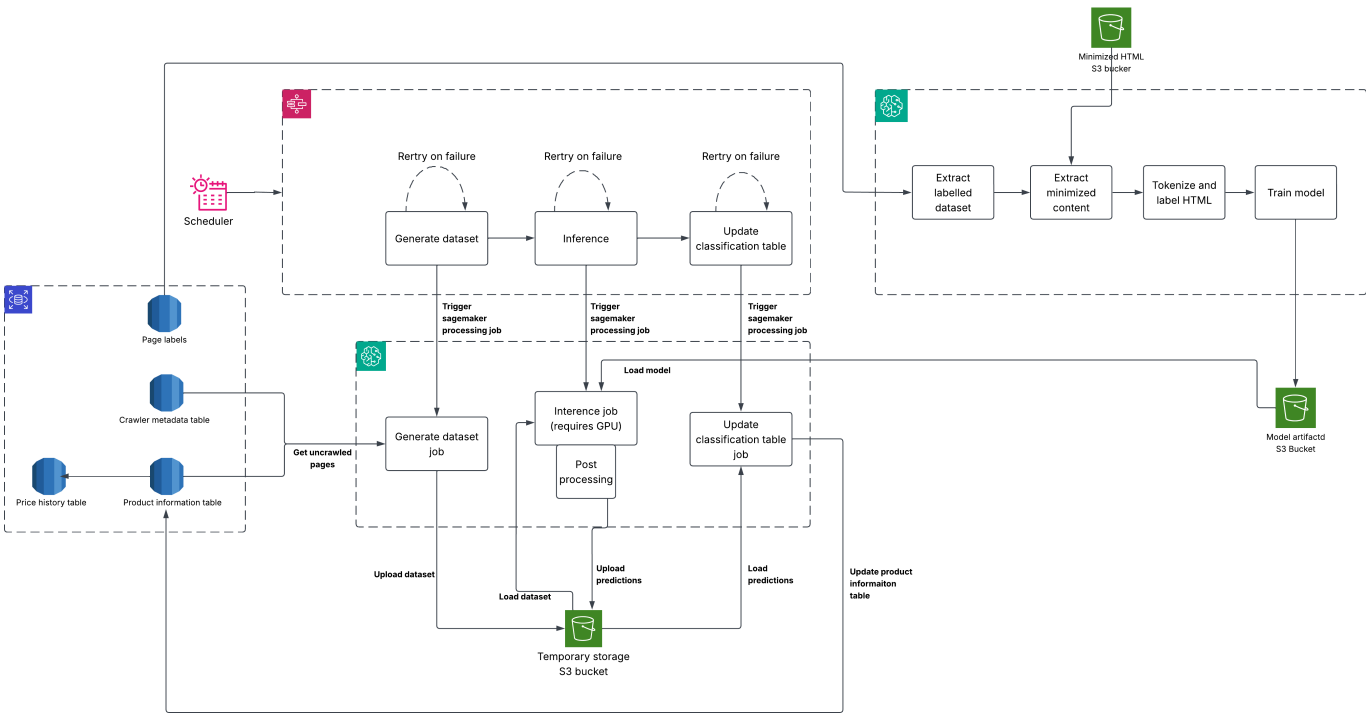

Figure 5.21: Overview of the information extraction module architecture.

5.7 Product matching

The product matching module is responsible for identifying and linking identical products across different websites. This is done in order to provide users with a view of the same product available on different websites, along with their respective prices and other relevant information. This task can be considered to be a document matching problem, where the goal is to determine whether two given product pages refer to the same product or not. The literature suggests that most document matching techniques rely on two phases: a candidate generation (filtering) phase and a candidate ranking (verification) phase. The candidate generation phase is a relatively computationally inexpensive process that aims to reduce the number of candidate pairs to be compared in the subsequent candidate ranking phase, which is a more computationally expensive process that aims to accurately determine whether the candidate pairs refer to the same product or not. Figure 5.22 illustrates how filtering and verification phases are used in the product matching module to identify and link identical products across different websites.

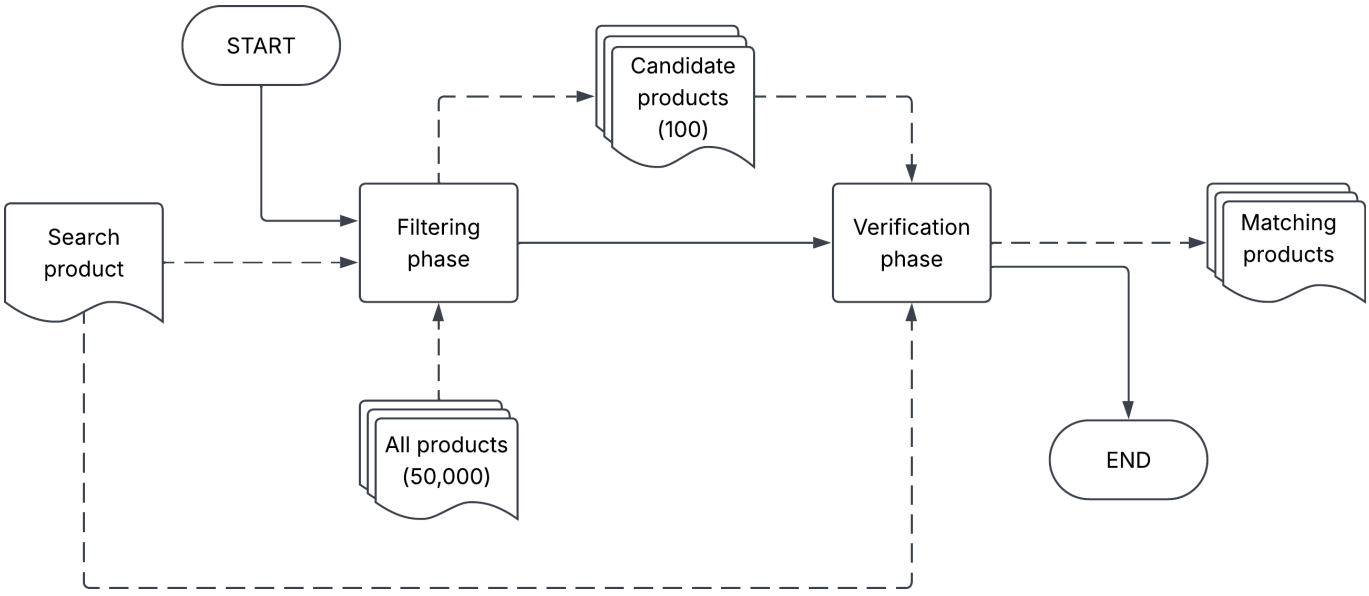

Figure 5.22: Overview of the product matching module.

5.7.1 Comparison of different approaches

Before deciding on the approaches to be used for the candidate generation and candidate ranking phases, a comparison of different approaches was conducted.

1. **Trigram similarity with title only:** This approach involves computing the trigram Jaccard similarity between the titles of the two product pages.
2. **Edit distance with title only:** This approach involves computing the edit distance between the titles of the two product pages.
3. **BM-25 algorithm:** This approach involves using the BM-25 algorithm to compute the similarity between the two product pages based on their titles and descriptions.
4. **RexBERT embeddings on title only (256 dimensions):** This approach involves using a pre-trained RexBERT model to generate embeddings for the titles of the two product pages, and then

computing the cosine similarity between the embeddings. RexBERT is a transformer-based model that is pre-trained on a large corpus of e-commerce product data, and is able to capture the semantic information in products.

5. **RexBERT embeddings on body (256 dimensions):** This approach involves using a pre-trained RexBERT model to generate embeddings for the bodies of the two product pages, and then computing the cosine similarity between the embeddings. This was chosen to investigate whether a model pre-trained on a large corpus of e-commerce product data could provide better performance other vector embedding models.

6. **RexBERT embeddings on body (768 dimensions):** This approach involves using a pre-trained RexBERT model to generate embeddings for the bodies of the two product pages, and then computing the cosine similarity between the embeddings.

7. **RexBERT embeddings on body (1024 dimensions):** This approach involves using a pre-trained RexBERT model to generate embeddings for the bodies of the two product pages, and then computing the cosine similarity between the embeddings.

8. **RexBERT embeddings on body converted to Markdown (768 dimensions):** This approach involves converting the bodies of the two product pages to Markdown format, generating embeddings using a pre-trained RexBERT model, and then computing the cosine similarity between the embeddings. This was chosen to investigate whether converting the bodies of the product pages to Markdown format could provide information in a more friendly format that could improve the performance of the product matching module.

9. **MarkupLM embeddings on body (768 dimensions):** This approach involves using a pre-trained MarkupLM model to generate embeddings for the bodies of the two product pages, and then computing the cosine similarity between the embeddings. This was chosen to investigate whether the HTML structure of the product pages could provide additional information that could improve the performance of the product matching module.

For all the vector-based approaches above, the cosine similarity was used as the similarity measure. The results of each of the above approaches are summarized in Table 5.3, which shows the compute required for generating and comparing the embeddings, the similarity scores for matching and non-matching products, the difference between the two similarity scores, and the performance difference between the two similarity scores.

Based on the these results, there were several observations that were made:

- The trigram similarity approach was found to be a good candidate for the candidate generation phase, as it was able to achieve a relatively high similarity score for matching products while being computationally inexpensive to compute.
- The BM-25 algorithm at first seemed to be ideal for the candidate ranking phase as it showed a very high difference between the similarity scores for matching and non-matching products, but it was found that this technique aggressively focussed on precision at the cost of recall, which is not ideal for the candidate ranking phase.
- The RexBERT embeddings on body (768 dimensions), while not the best performing approach overall, showed the most promise out of the vector embedding approaches.

Technique	Compute for generation	Compute for comparison	Matching products	Non-matching products	Difference	Percentage difference
Trigram similarity with title only	N/A	VERY LOW	0.73	0.52	0.21	38.33%
Edit distance with title only	N/A	VERY LOW	0.66	0.57	0.09	16.57%
BM-25 algorithm	MEDIUM	LOW	0.58	0.10	0.48	473.69%
RexBERT embeddings on title only (256 dimensions)	LOW	LOW	0.96	0.93	0.02	1.78%
RexBERT embeddings on body (256 dimensions)	MEDIUM	MEDIUM	0.98	0.97	0.01	1.03%
RexBERT embeddings on body (768 dimensions)	HIGH	MEDIUM	0.92	0.86	0.06	6.98%
RexBERT embeddings on body (1024 dimensions)	HIGH	HIGH	0.93	0.93	0.00	0.00%
RexBERT embeddings on body converted to markdown (768 dimensions)	HIGH	MEDIUM	0.91	0.88	0.03	3.41%
MarkupLM embeddings on body (768 dimensions)	VERY HIGH	MEDIUM	0.97	0.93	0.04	4.30%

Table 5.3: Comparison of different product matching approaches

None of the embedding models showed great performance as these models were not pre-trained on web-based corpora. Even for models like RexBERT which are pre-trained trained on e-commerce data, distinguishing between matching and non-matching product pairs were difficult as the overall size of e-commerce products is very large. The same product model with slightly different variations (e.g. colour, storage, memory), or even two different models, but very similar characteristics (e.g. high-end flagship smartphones) would appear very close together in the vector space. Furthermore, the models were more likely to place products from the same site close together due to the common features/patterns that are available in the pages themselves and how they choose to organize information.

5.7.2 Candidate generation (filtering) phase

During this phase, when given a product, the goal is to generate a relatively small number of candidate pairs that are likely to refer to the same product. There are two main criteria to be considered when designing the candidate generation phase:

- High recall: The candidate generation phase should aim to generate as many true positive pairs as

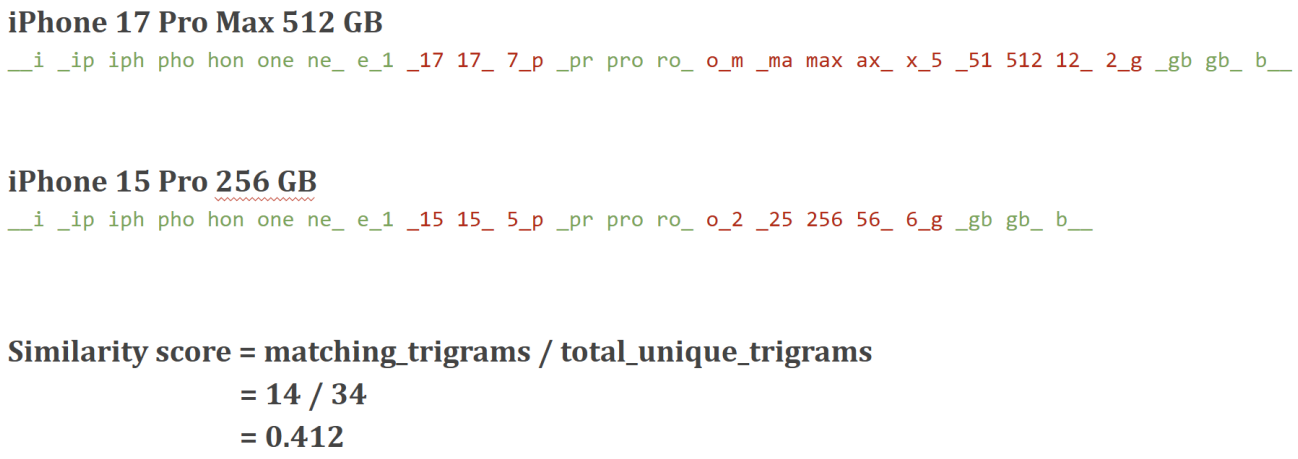

Figure 5.23: Example of trigram Jaccard similarity between two product titles.

possible, even if it means generating some false positive pairs. This is because the subsequent candidate ranking phase will be responsible for filtering out the false positives.

- Low computational cost: The candidate generation phase should be computationally inexpensive, as it will be applied to a large number of products and candidate pairs.

Considering the types of data available at this point, the technique opted for was to use the names of the products to generate candidate pairs. For a given product, another page was considered be a matching candidate if the trigram Jaccard similarity between the two product names was greater than a certain threshold. This specific metric was chosen because it is relatively computationally inexpensive to compute, and it is able to capture the similarity between two strings even if they have minor differences in spelling or formatting. Moreover, there is built-in support for computing the trigram similarity on PostgreSQL via the `pg_trgm` extension, which allows for efficient computation of the similarity between two strings. This also had the added benefit of allowing the candidate generation phase to be implemented as a SQL query, which is relatively easy to implement and maintain, without the need for additional infrastructure or services which helped in keeping the system simple and cost-effective.

Trigram Jaccard similarity is a measure of similarity between two strings, which is defined as the size of the intersection of the sets of trigrams (three-character sequences) in the two strings divided by the size of the union of the sets of trigrams in the two strings. An example of this is illustrated in Figure 5.23.

In order to allow the maximum flexibility in terms of the threshold to be used for generating candidate pairs, the threshold was made configurable, but a default value of 0.3 was used, which was found to be a good balance between recall and computational cost.

5.7.3 Candidate ranking (verification) phase

During this phase, the goal is to accurately determine whether the candidate pairs generated in the previous phase refer to the same product or not. This is a more computationally expensive process, as it involves comparing the candidate pairs using more sophisticated techniques, such as machine learning models or vector-based similarity measures. For this phase, both precision and recall are important, as the goal is to accurately identify the true positive pairs while minimizing the number of false positives.

The literature suggested two main approaches for the candidate ranking phase: a machine learning approach and a vector-based approach. A machine-learning based approach involves training a machine learning model to classify the candidate pairs as either matching or non-matching, based on a set of features extracted from the product pages. This approach was considered to be infeasible as it would require the model to be online at all times, which would have been expensive. Unlike the page classification and information extraction modules, it was also not possible to have pre-computed results for all product pairs as the number of pairs would be exponential in the number of products, which would have been infeasible to store and maintain.

Therefore, a vector-based approach was chosen, which involves representing the product pages as vectors in a high-dimensional space and computing the similarity between the vectors to determine whether the candidate pairs refer to the same product or not. Vector similarity measures such as cosine similarity or Euclidean distance can be used to compute the similarity between the vectors, and a threshold can be set to determine whether the candidate pairs are considered to be matching or not.

This process can also be implemented in three phases.

1. **Fine-tuning phase:** In this phase, a pre-trained language model is fine-tuned on a labelled dataset of positive and negative product pairs to learn a vector representation of the product pages that captures the semantic similarity between them.
2. **Vector generation phase:** In this phase, the fine-tuned model is used to generate vector representations for all product pages in the database.
3. **Matching phase:** In this phase, the vector representations of the candidate pairs are compared using a similarity measure, and a threshold is set to determine whether the candidate pairs are considered to be matching or not.

Fine-tuning phase

As shown in Table 5.3, while the embedding models did not show great performance in distinguishing between matching and non-matching products, these models can be improved using contrastive fine-tuning, which is a technique that involves training the model to minimize the distance between the embeddings of matching product pairs while maximizing the distance between the embeddings of non-matching product pairs. This is done by providing the model with:

- **Soft positive pairs:** These are pairs of product pages that are considered to be matching, but the original embeddings of the product pages are not very close to each other in the embedding space.
- **Hard negative pairs:** These are pairs of product pages that are considered to be non-matching, but the original embeddings of the product pages are very close to each other in the embedding space.

By providing the model with these types of pairs, it is able to learn a more robust representation of the product pages that captures the semantic similarity between them, which can improve the performance of the product matching module. It would more effectively learn to generate embeddings by prioritizing more discriminative features that are relevant for distinguishing between matching and non-matching products, which can improve the performance of the product matching module. Such features can include the brand, model number, specifications, and other relevant attributes of the products, which can help the model to better capture the semantic similarity between the product pages. Figure 5.24 illustrates how contrastive

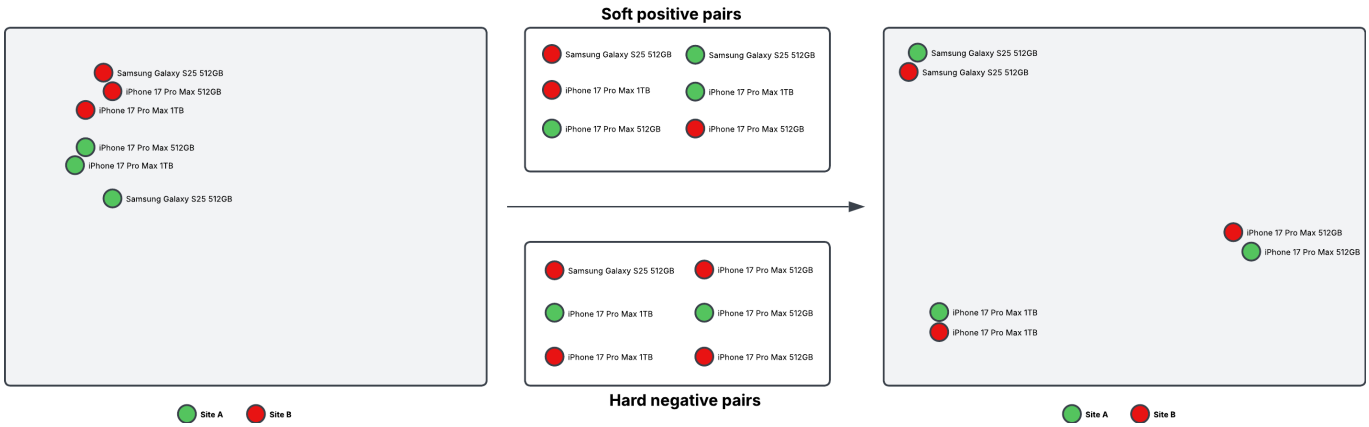

Figure 5.24: How contrastive fine-tuning can bring similar products closer together in the embedding space, while pushing dissimilar products further apart.

fine-tuning can bring similar products closer together in the embedding space, while pushing dissimilar products further apart.

Nearly 2000 product pairs were manually labelled as either matching or non-matching, and these were used to fine-tune the pre-trained RexBERT model on the body of each page's minimized content. These pairs were selected to be a mix of both soft positive pairs and hard negative pairs, in order to provide the model with a diverse set of examples to learn from. Using a similar process as in the page classification model and the information extraction model, the fine-tuned model was trained on AWS Sagemaker using their feature for hyperparameter tuning, which allowed for the automatic selection of the best hyperparameters for the model.

Using this approach, the model was able to learn a more robust representation of the product pages that captures the semantic similarity between them, which improved the performance of the product matching module. When using the embeddings generated by this fine-tuned model, the average similarity scores for matching and non-matching products were `0.896` and `0.356` respectively, with a difference of `0.540` and a percentage difference of `151.69%`, which was a significant improvement over the original embeddings generated by the pre-trained model, which had a difference of only `0.06` and a percentage difference of `6.98%`. This shows that the fine-tuned model was able to learn a more discriminative representation of the product pages that captures the semantic similarity between them, which improved the performance of the product matching module.

Vector generation phase

In this phase, the fine-tuned model is used to generate vector representations for all product pages in the database. This is done by passing the text content of the minimized HTML of each product page through the fine-tuned model to obtain the corresponding embedding vector.. These embedding vectors are then stored in the PostgreSQL database. PostgreSQL allows for the storage of vector data types, using the `pgvector` extension, which allows for efficient storage, retrieval and comparison of vector data. Figure 5.25 illustrates how the vector generation phase is used to generate vector representations for all product pages in the database.

Even though the vector generation phase is a computationally expensive process, it is only done once for each product page, and the resulting embeddings can be reused for all subsequent candidate pairs that

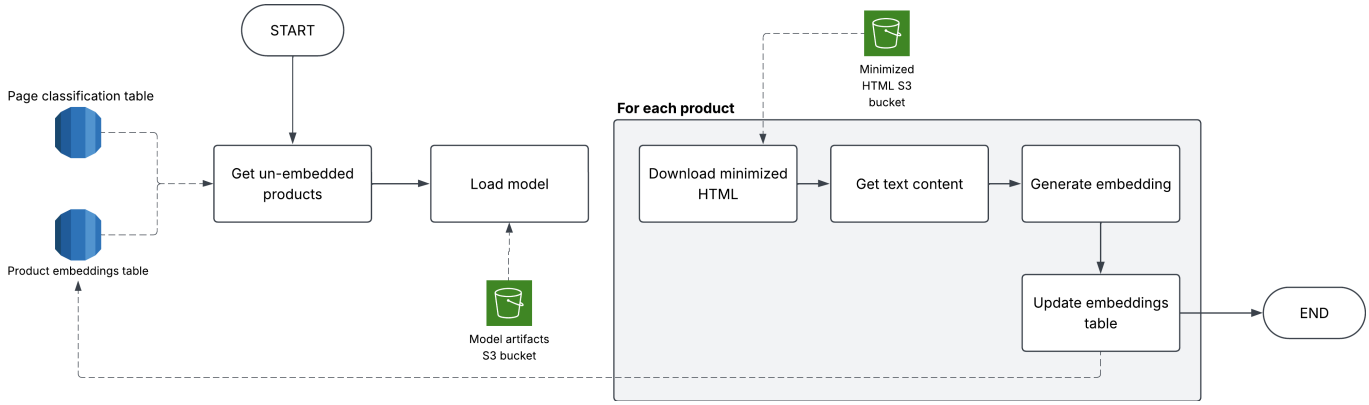

Figure 5.25: Overview of the vector generation phase.

involve that product page. Therefore, the computational cost of this phase is spread out over all subsequent candidate pairs that involve that product page, which makes it a relatively efficient process in the long run.

Matching phase

In this phase, the vector representations of the candidate pairs are compared using cosine-similarity, and a threshold is set to determine whether the candidate pairs are considered to be matching or not. This could be done entirely within the PostgreSQL database, using the `pgvector` extension to compute the cosine similarity between the embedding vectors of the candidate pairs, and then applying a threshold to determine whether the candidate pairs are considered to be matching or not. Different values for this threshold were experimented with, and `0.691` was found to provide the highest F1 score. However, this threshold was left to be configurable to be able to adjust it based on user preferences, as some users may prefer to have a higher precision at the cost of recall, while others may prefer to have a higher recall at the cost of precision.

5.8 Proof-of-concept (PoC) system

The proof-of-concept (PoC) system is a web-based application that allows users to search for products, view their price history, and compare prices across different websites. It includes a user interface which provides the following features:

- A search bar that allows users to enter a product name or keyword to search for products across different websites.
- A product list page that displays the search results, including product name, price, image, and stock status.
- A product detail page that displays the product's price history, including a graph of the price changes over time and a table of historical prices.
- A similar products section that displays a list of identical products from different websites, allowing users to compare prices and make informed purchasing decisions.

- A metadata section that displays additional information about the product page's lifecycle, which includes the dates and times of the last crawl, mininimization, classification and information extraction.
- A side-by-side view of the original and minimized HTML content of the product page, allowing users to see the differences between the two versions.

The front-end of the application is build using Next.Js, a popular framework for building web applications, and is hosted using AWS Amplify, a cloud-based platform for building and deploying web applications. The back-end (API) is built using Golang with the Mux router, a lightweight and efficient HTTP router for building RESTful APIs. It is hosted using AWS Lambda, a serverless computing platform that allows for scalable and cost-effective deployment of APIs. The API is fronted by an AWS API Gateway, which provides a secure and scalable entry point for the API, and also handles rate limiting to prevent abuse and ensure fair usage of the system.

The API provides the following endpoints:

- `GET /products?url=url` - Retrieves product information based on the provided URL.
- `GET /products/search?query=query` - Searches for products based on the provided query.
- `GET /products/history?url=url` - Retrieves the price history of a product based on the provided URL.
- `GET /products/similar?url=url` - Retrieves a list of similar products based on the provided URL.
- `GET /products/metadata?url=url` - Retrieves metadata information about the product page's lifecycle based on the provided URL.
- `GET /products/source-crawled-page?url=url` - Retrieves the original crawled HTML content of the product page based on the provided URL.
- `GET /products/source-minimized-page?url=url` - Retrieves the minimized HTML content of the product page based on the provided URL.
- `GET /products/random` - Retrieves a random product from the database.
- `POST /products/mark-matching?url1=url1&url2=url2&isMatching=match` - Marks two products as matching based on the provided URLs and match status.

Figure 5.26 illustrates the architecture of the proof-of-concept (PoC) system, showing the interactions between the front-end, back-end, and various AWS services used for hosting and deployment.

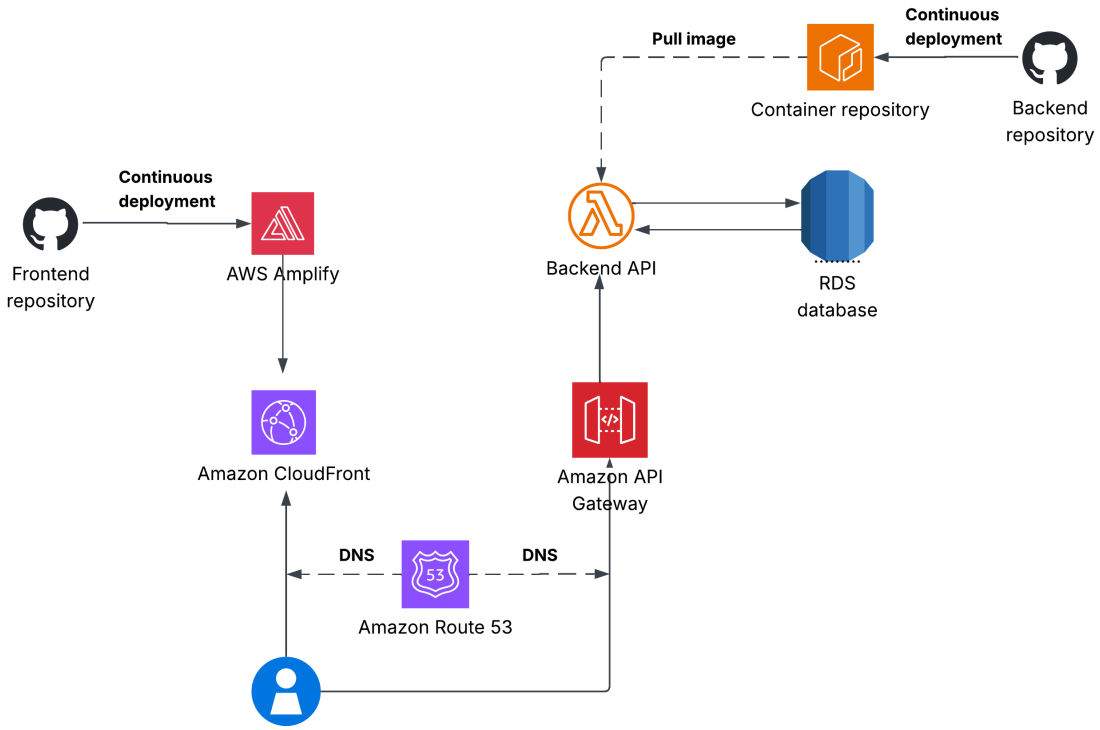

Figure 5.26: Proof-of-concept (PoC) system architecture